

第三章

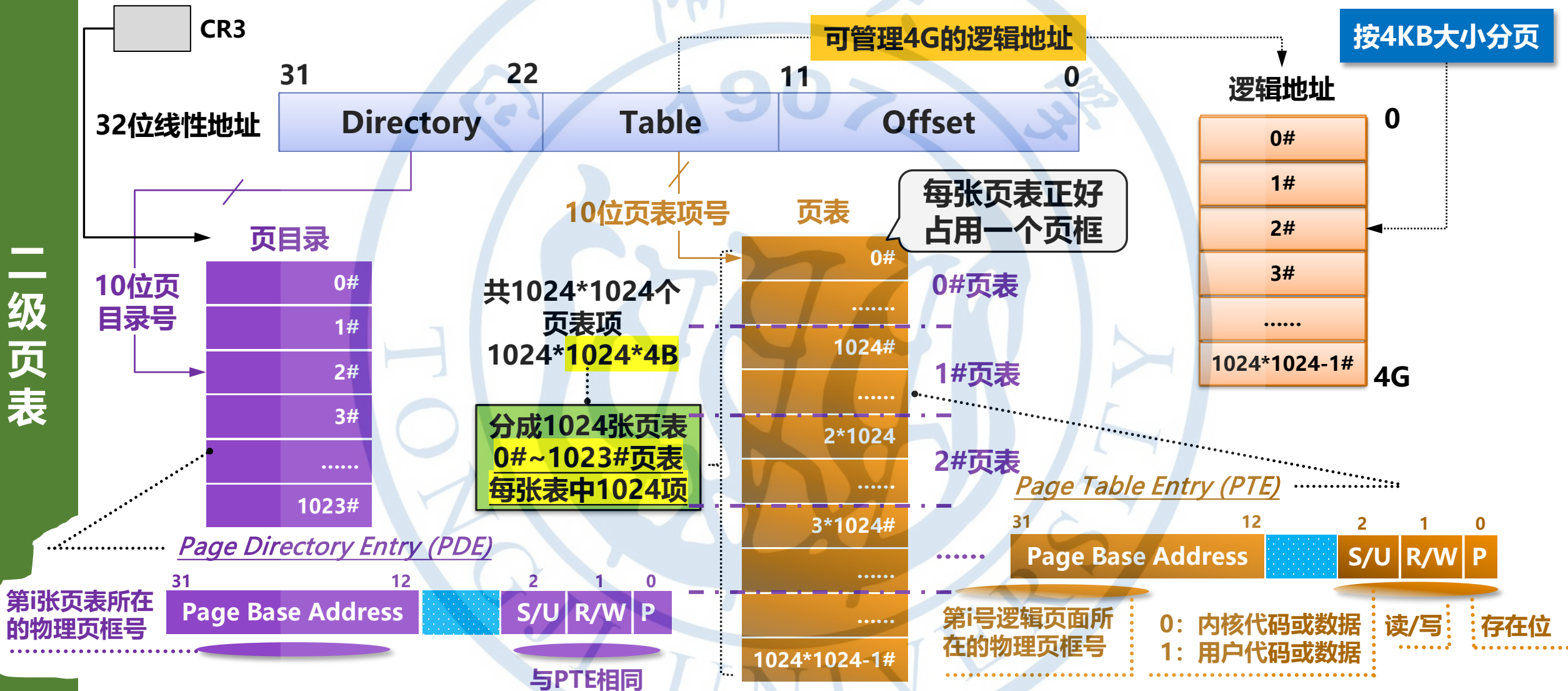
存储管理



i386线性地址空间



二级页表





UNIX V6++的地址变换



汇编语言

C语言

实模式

BIOS自检程序

各种检测;
初始化;
查找引导设备
查找引导扇区
加载boot程序

Jmp 0000:7C00

设置GDT;
进入保护模式;
设置段寄存器;
加载内核到1M
起的物理内存;

跳转到内核

保护模式

开启分页的保护模式

16位偏移地址寄存器

20位内存
物理地址

1M

物理地址

+

org 0x7c00

*16 (左移4位)

16位段寄存器

从逻辑地址到物理地址



UNIX V6++的地址变换



汇编语言

C语言

实模式

BIOS自检程序

各种检测;
初始化;
查找引导设备
查找引导扇区
加载boot程序

Jmp 0000:7C00

设置GDT;
进入保护模式;
设置段寄存器;
加载内核到1M
起的物理内存;
跳转到内核

1

1

保护模式

32位线性地址:

0x00000000 + 32位偏移地址

org 0x7c00

开启分页的保护模式

1M 线性地址

未开启分页, 用线性地址访问内存

1M 物理地址

CS = 0x8

起始地址	限长
0x00000000	0
0x00000000	0xFFFF
0x00000000	0xFFFF
0x40000000	0xFFFF
0x40000000	0xFFFF

需要线性地址在数值上 = 物理地址
即: 事实上关闭分段

从逻辑地址到物理地址



UNIX V6++的地址变换



汇编语言

C语言

实模式

BIOS自检程序

各种检测;
初始化;
查找引导设备
查找引导扇区
加载boot程序

Jmp 0000:7C00

设置GDT;
进入保护模式;
设置段寄存器;
加载内核到1M
起的物理内存;

跳转到内核

2

1

保护模式

2

开启分页的保护模式

32位线性地址:

$0x00000000 + 32\text{位偏移地址}$

32位线性地址:

$0x40000000 + 32\text{位偏移地址}$

org 0x7c00

.text 0xc0100000:

内核入口的逻辑地址

线性地址

1M

3G

3G + 1M

CS = 0x18

起始地址

限长

0x00000000

0

0x00000000

0xFFFF

0x00000000

0xFFFF

0x40000000

0xFFFF

0x40000000

0xFFFF

未开启分页, 用线性地址访问内存

要借助分段机制将3G + 1M的
逻辑地址修正到1M的物理地址

1M

物理地址

从逻辑地址到物理地址



UNIX V6++的地址变换



汇编语言

C语言

实模式

1

保护模式

2

开启分段的保护模式

3

32位线性地址:

$0x00000000 + 32\text{位偏移地址}$

32位线性地址:

$0x40000000 + 32\text{位偏移地址}$

32位线性地址:

$0x00000000 + 32\text{位偏移地址}$

各种检测;
初始化;
查找引导设备
查找引导扇区
加载boot程序

`Jmp 0000:7C00`

设置GDT;
进入保护模式;
设置段寄存器;
加载内核到1M
起的物理内存;
跳转到内核

`org 0x7c00`

`.text 0xc0100000:`

内核入口的逻辑地址

线性地址

1M

3G

3G + 1M

$CS = 0x8$

起始地址

限长

0x00000000

0

0x00000000

0xFFFF

0x00000000

0xFFFF

0x40000000

0xFFFF

0x40000000

0xFFFF

线性地址转换成物理地址访问内存

事实上关闭分段, 要通过分页机制完成地址变换

物理地址

1M

主要内容

3.1 存储管理的主要任务

3.2 连续分配方式

3.3 页式存储管理

3.4 段式与段页式存储管理**

3.5 **UNIX 存储管理**

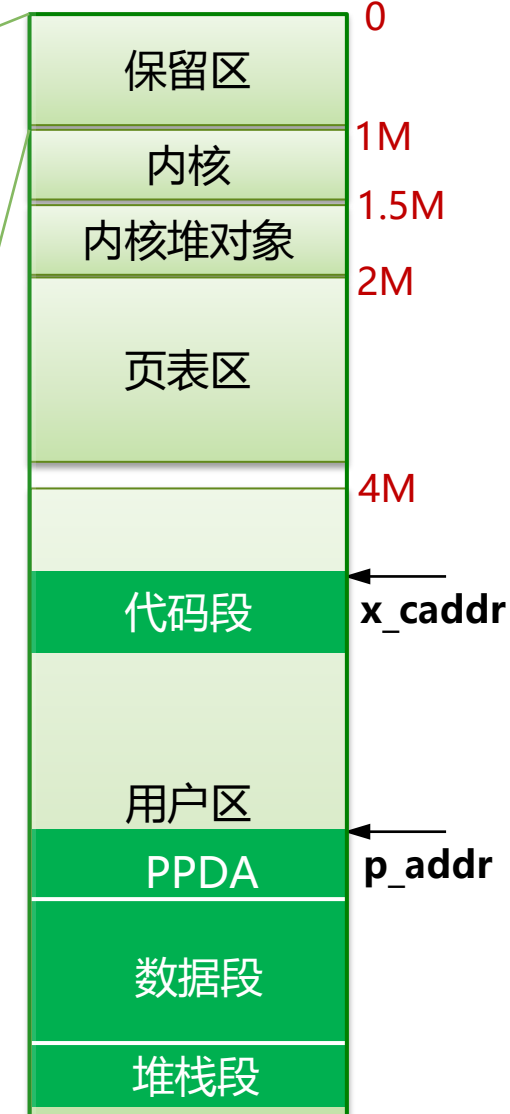
- 物理地址空间
- 程序地址空间
- 地址变换
- 存储空间管理



UNIX V6++物理地址空间

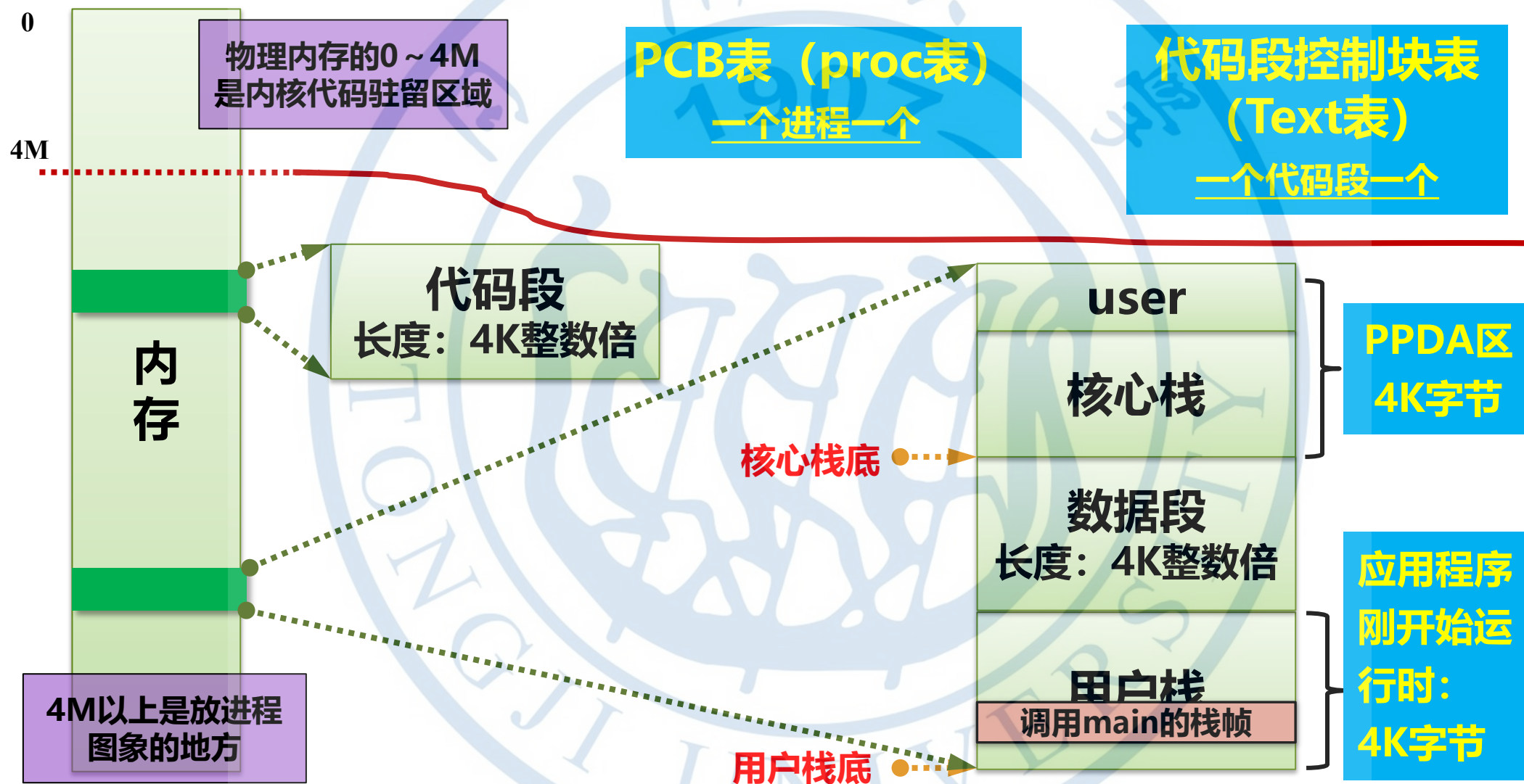


起始地址	大小	用途
0x00000	1KB	BIOS的中断向量表
0x00400	256B	BIOS数据区
0x00500	29KB+768B	可用区域
0x07C00	512B	引导扇区数据，被BIOS加载到此处
0x07E00	607KB+512B	可用区域
0x9FC00	1KB	扩展BIOS数据区
0xA0000	64KB	彩色显示适配器缓存
0xB0000	32KB	黑白显示适配器缓存
0xB8000	32KB	文本模式显示适配器缓存
0xC0000	32KB	显示适配器BIOS
0xC8000	160KB	映射硬件适配器ROM或内存映射式的IO
0xF0000	64KB	BIOS程序，入口地址在0xFFFF0



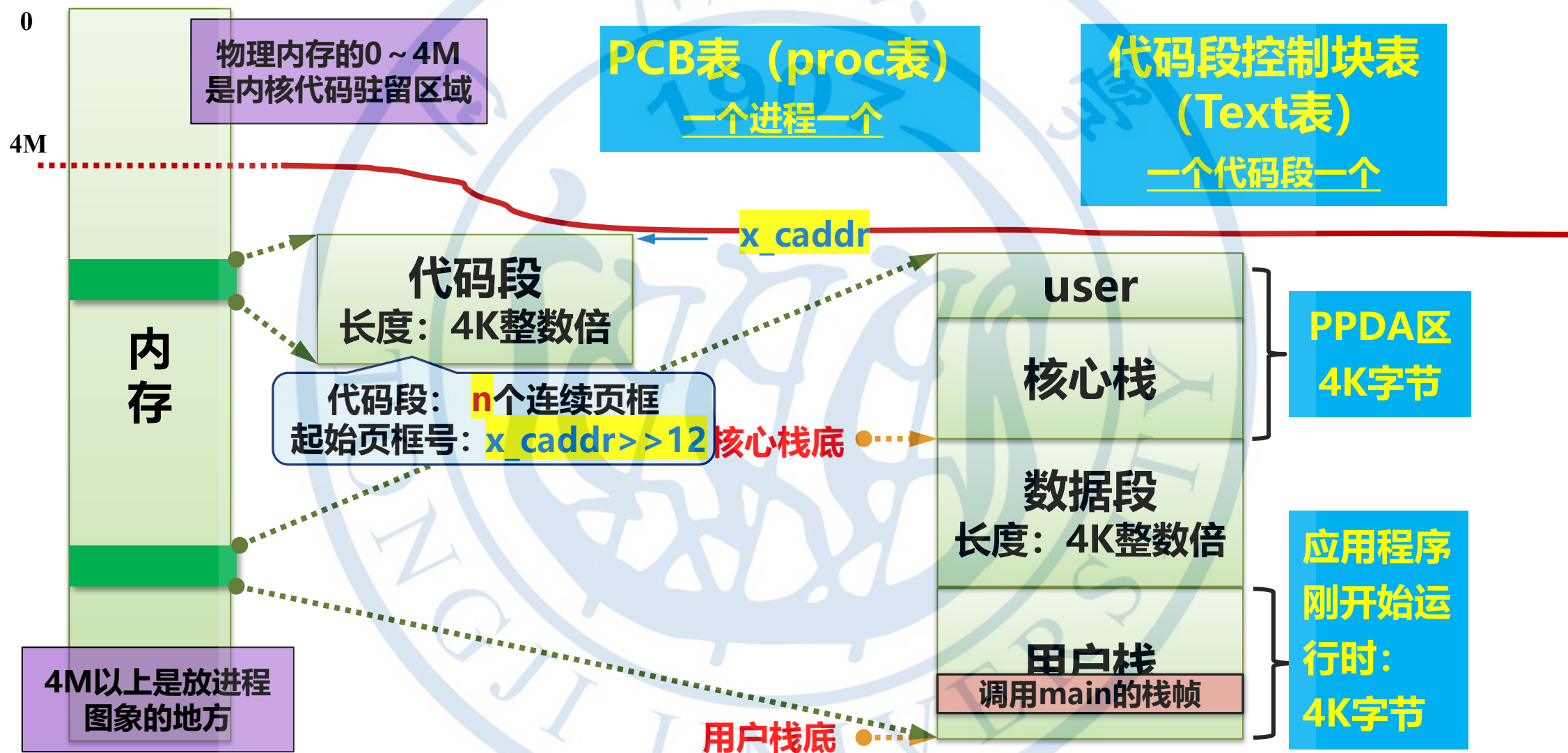


UNIX V6 + 物理地址空间



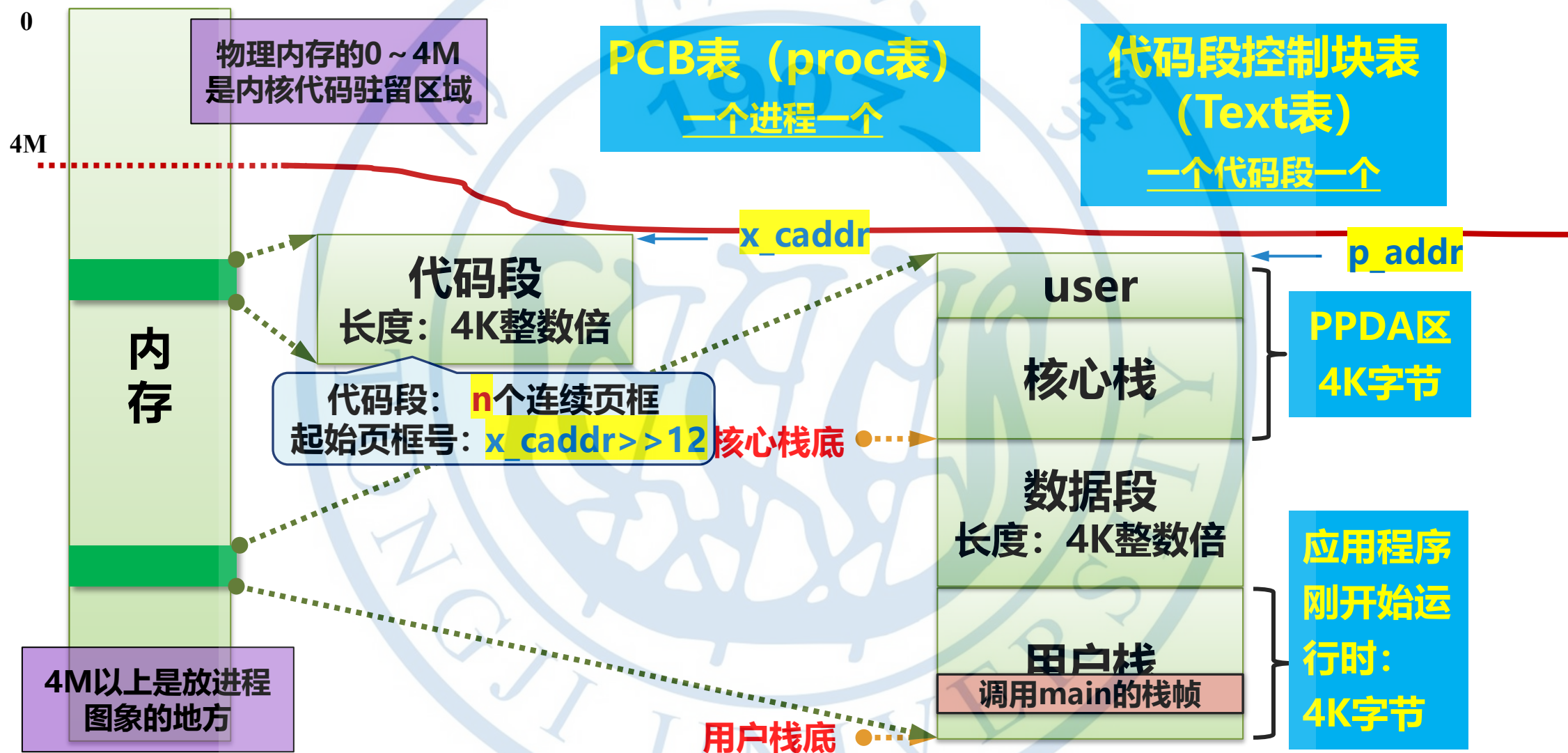


UNIX V6++物理地址空间



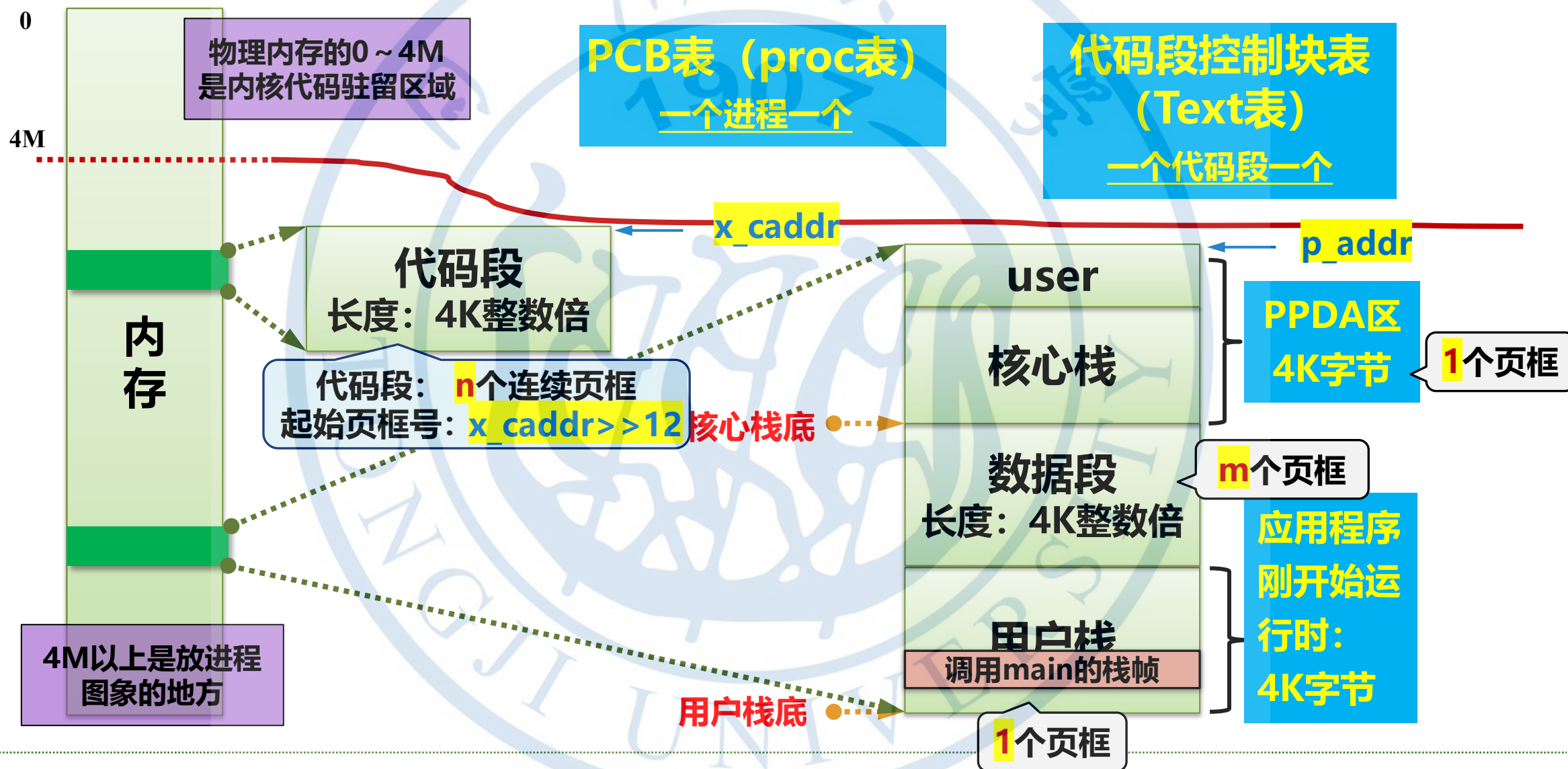


UNIX V6++ 物理地址空间



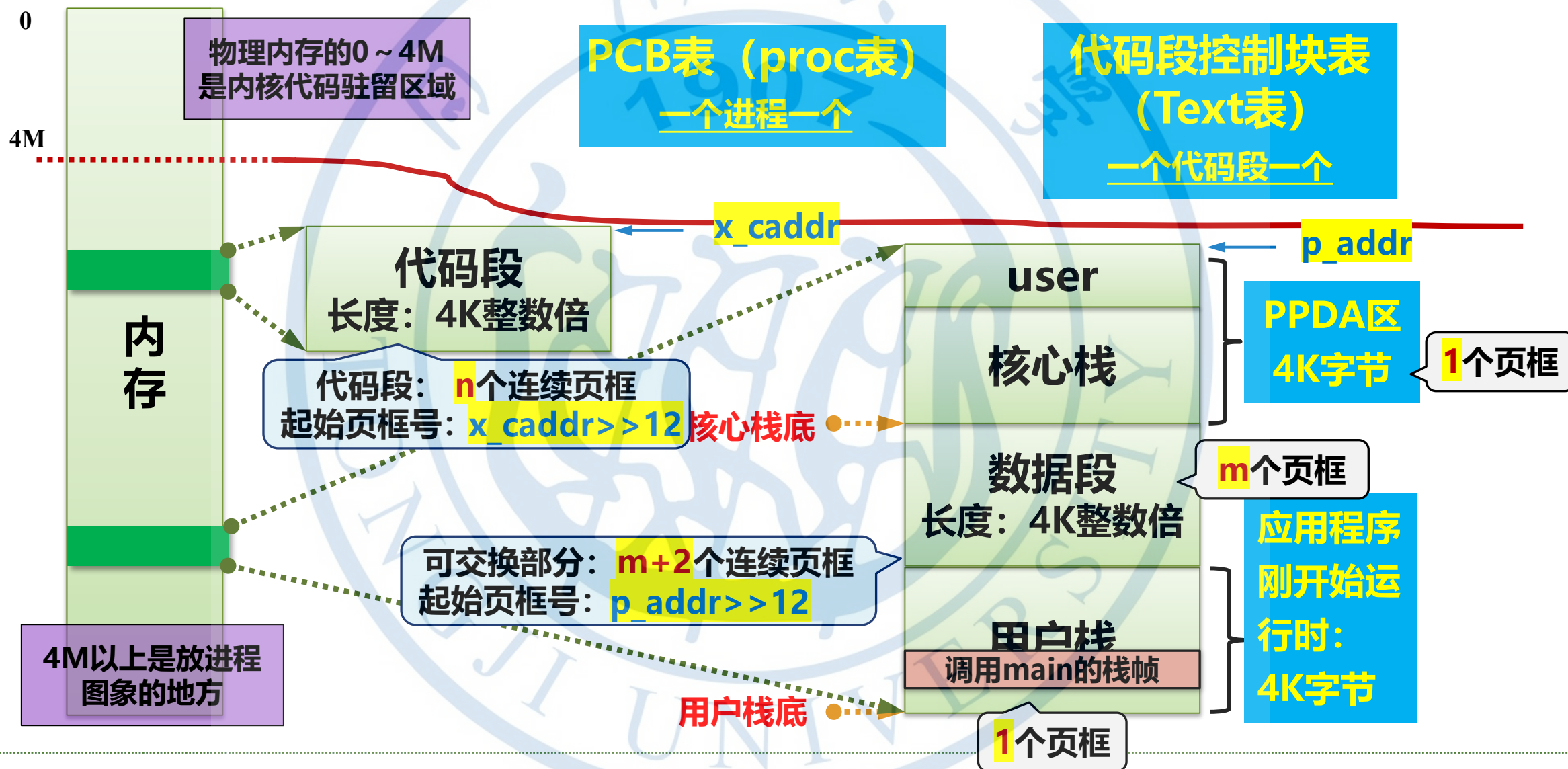


UNIX V6++ 物理地址空间





UNIX V6++ 物理地址空间



主要内容

3.1 存储管理的主要任务

3.2 连续分配方式

3.3 页式存储管理

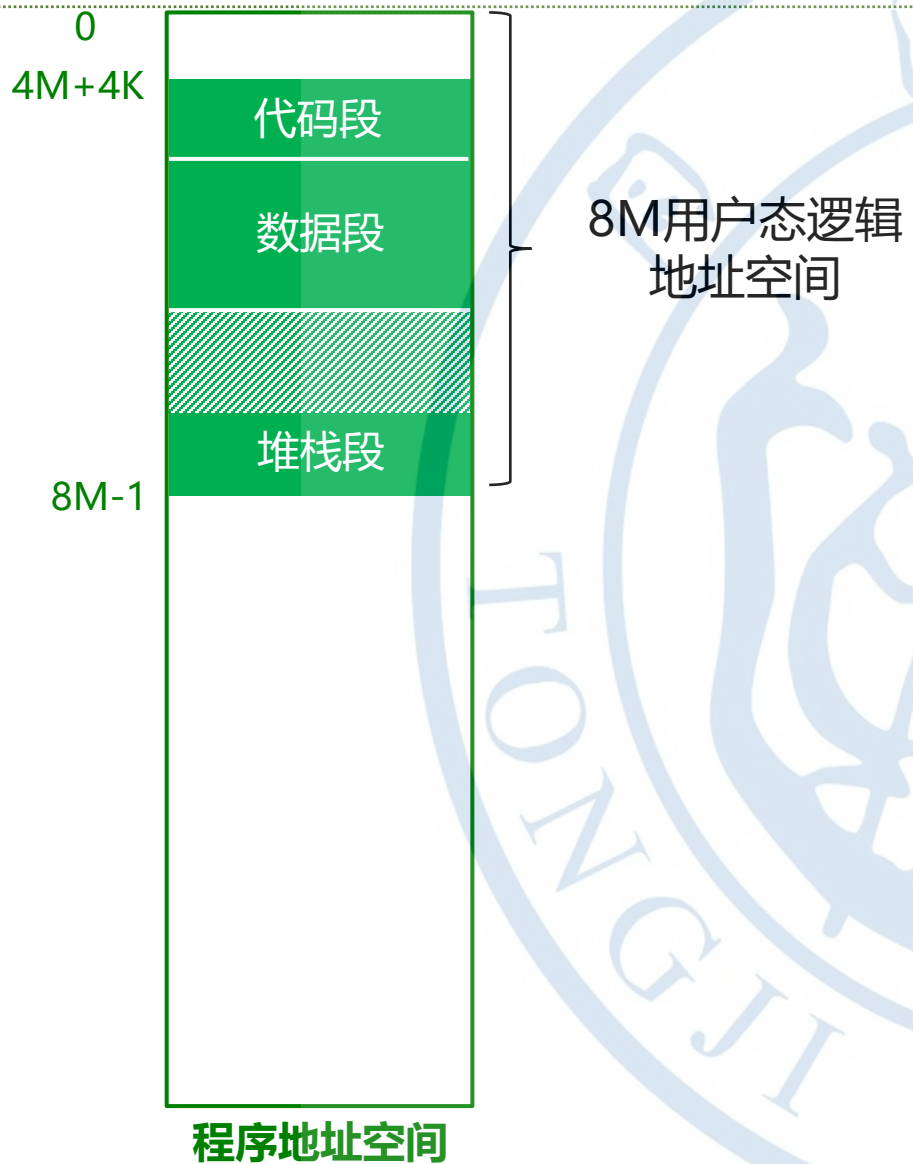
3.4 段式与段页式存储管理**

3.5 **UNIX 存储管理**

- 物理地址空间
- **程序地址空间**
- 地址变换
- 存储空间管理



UNIX V6++的地址变换





UNIX V6++的地址变换



页表的构成

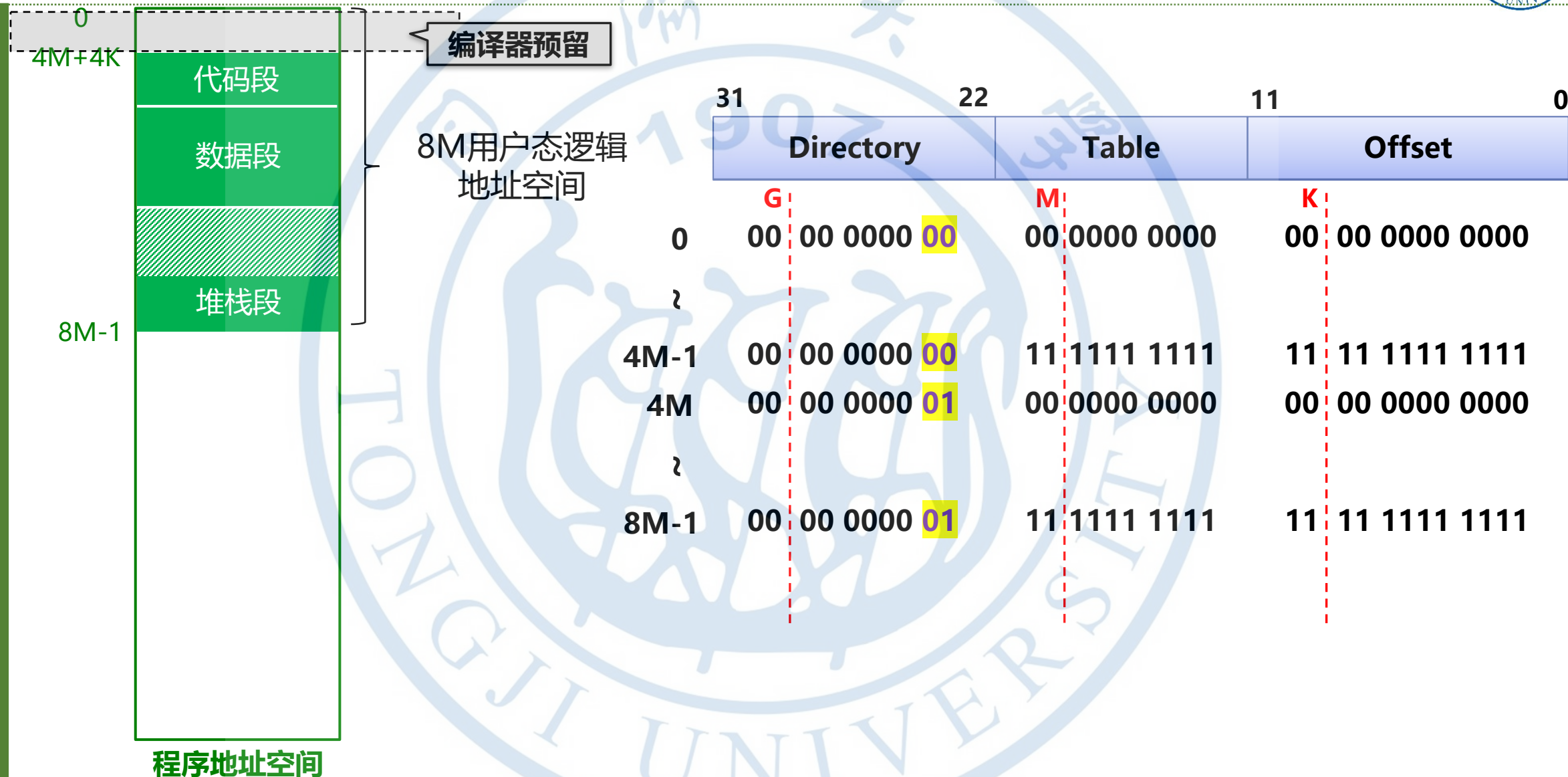




UNIX V6++的地址变换



页表的构成

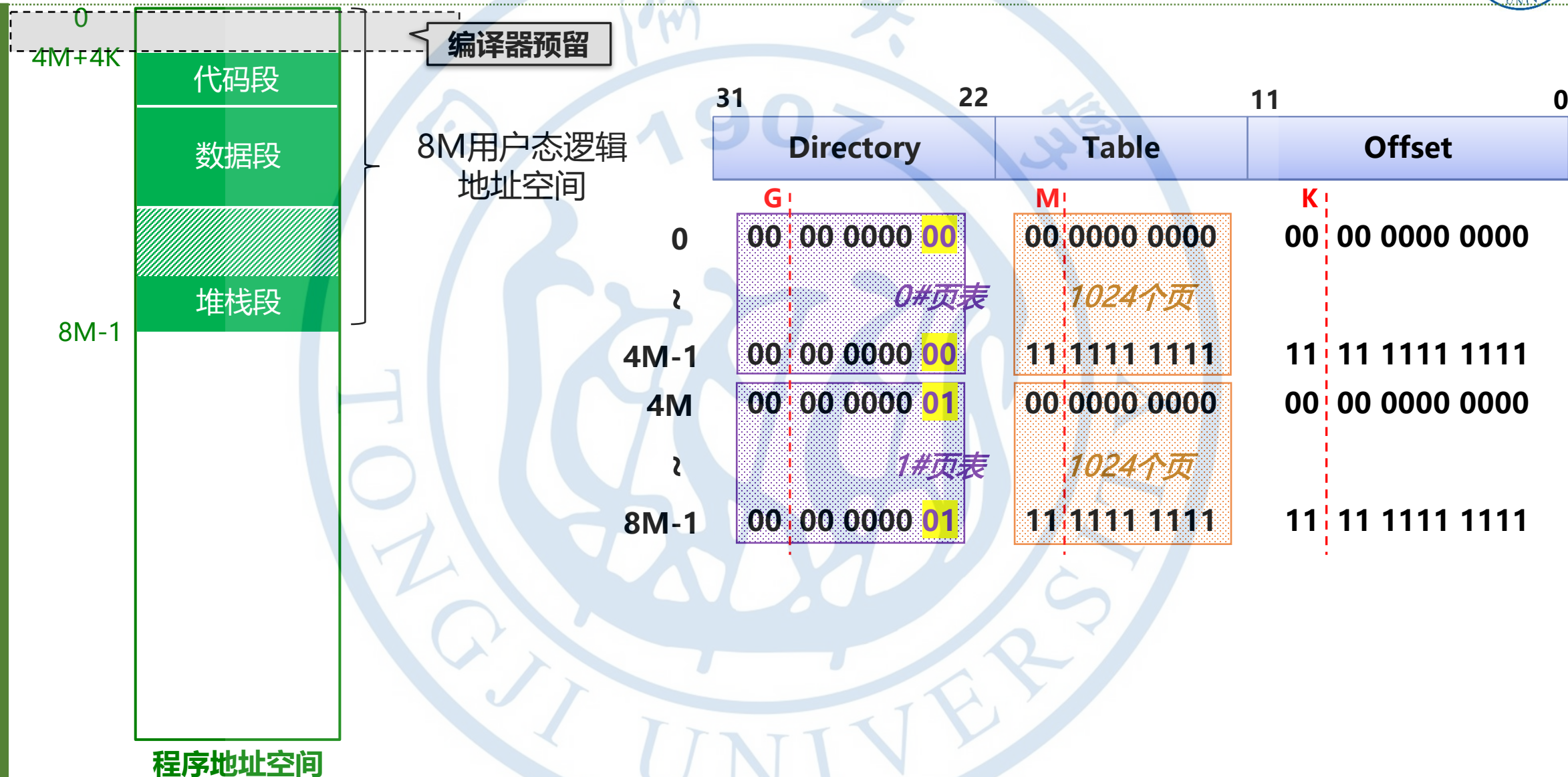




UNIX V6++的地址变换



页表的构成

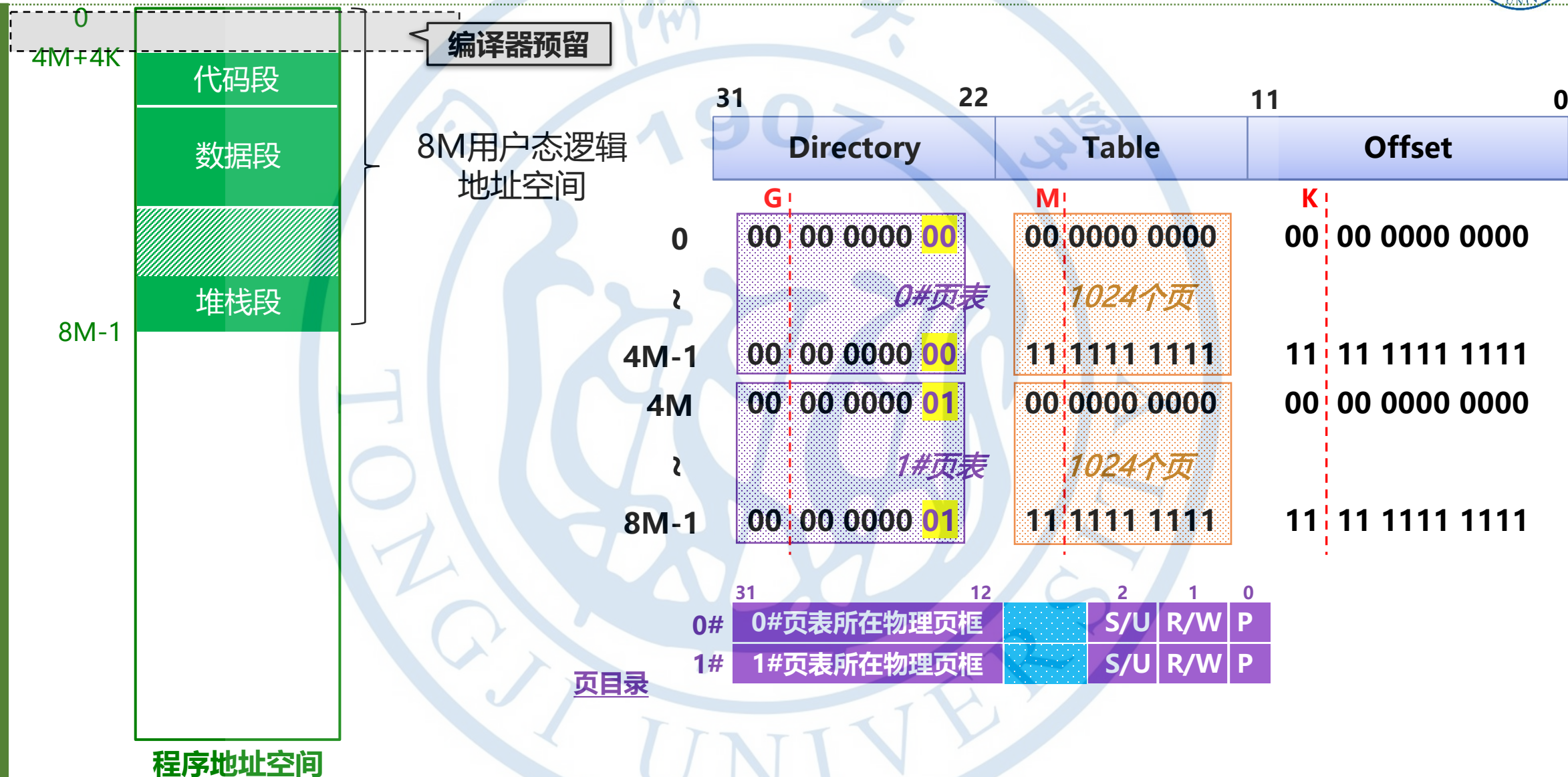




UNIX V6++的地址变换



页表的构成





UNIX V6++的地址变换

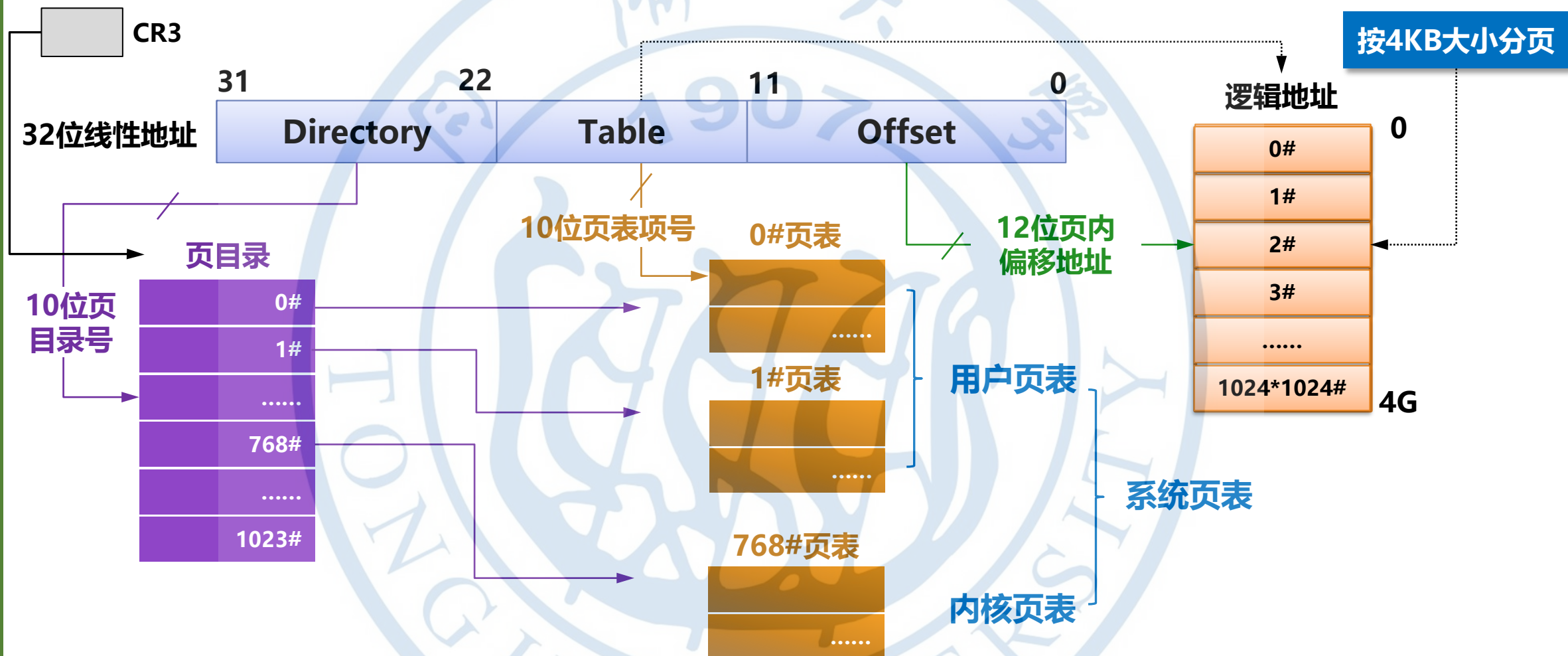


页表的构成





UNIX V6++ 程序地址空间





存储空间管理



二级页表

```
struct PageDirectoryEntry /* 页目录中的每一个表项 */
```

```
{  
    unsigned char m_Present : 1;  
    unsigned char m_ReadWriter : 1;  
    unsigned char m_UserSupervisor : 1;  
    unsigned char m_WriteThrough : 1;  
    unsigned char m_CacheDisabled : 1;  
    unsigned char m_Accessed : 1;  
    unsigned char m_Reserved : 1;  
    unsigned char m_PageSize : 1;  
    unsigned char m_GlobalPage : 1;  
    unsigned char m_ForSystemUser : 3;  
    unsigned int m_PageTableBaseAddress : 20;  
}__attribute__((packed));
```

```
class PageDirectory
```

```
{  
public:  
    .....;  
    PageTable& GetPageTableByIdx(int idx);  
    PageDirectoryEntry& GetPageDirectoryEntryByIdx(int idx);  
  
public:  
    PageDirectoryEntry m_Entrys[1024];  
};
```





存储空间管理



二级页表

```
struct PageTableEntry /* 页目录中的每一个表项 */
```

```
{
    unsigned char m_Present : 1;
    unsigned char m_ReadWriter : 1;
    unsigned char m_UserSupervisor : 1;
    unsigned char m_WriteThrough : 1;
    unsigned char m_CacheDisabled : 1;
    unsigned char m_Accessed : 1;
    unsigned char m_Dirty : 1;
    unsigned char m_PageTableAttributeIndex : 1;
    unsigned char m_GlobalPage : 1;
    unsigned char m_ForSystemUser : 3;
    unsigned int m_PageBaseAddress : 20;
}__attribute__((packed));
```

```
class PageTable
```

```
{
public:
    static const unsigned int ENTRY_CNT_PER_PAGETABLE = 1024; /* 每张页表有1024个表项 */
    static const unsigned int SIZE_PER_PAGETABLE_MAP = 0x400000; /* 每张页表可管理4M地址 */
    .....;
public:
    PageTableEntry m_Entrys[ENTRY_CNT_PER_PAGETABLE];
};
```

Page Table Entry (PTE)



第i号逻辑页面所
在的物理页框号

0: 内核代码或数据
1: 用户代码或数据

读/写

存在位

主要内容

3.1 存储管理的主要任务

3.2 连续分配方式

3.3 页式存储管理

3.4 段式与段页式存储管理**

3.5 **UNIX 存储管理**

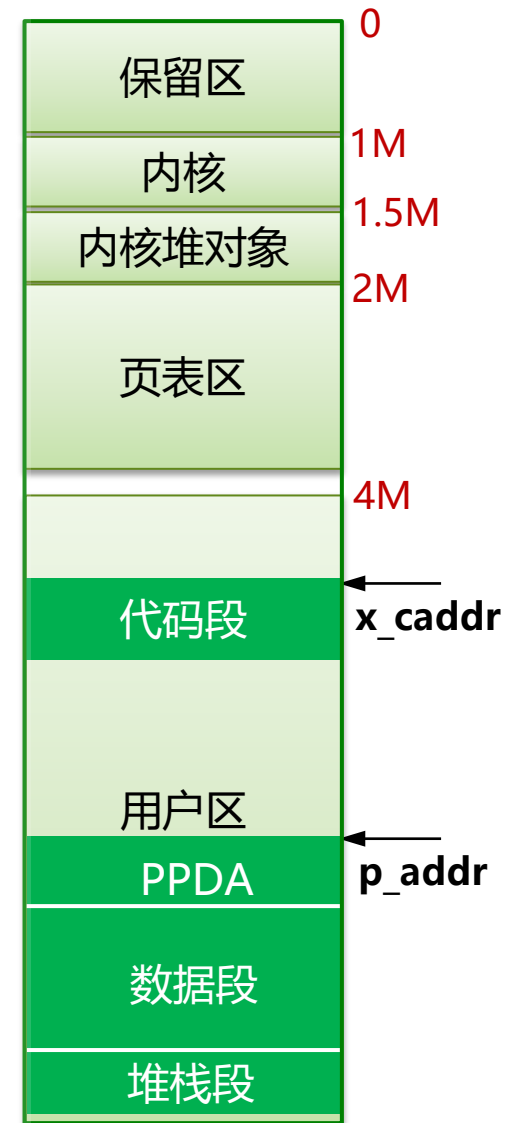
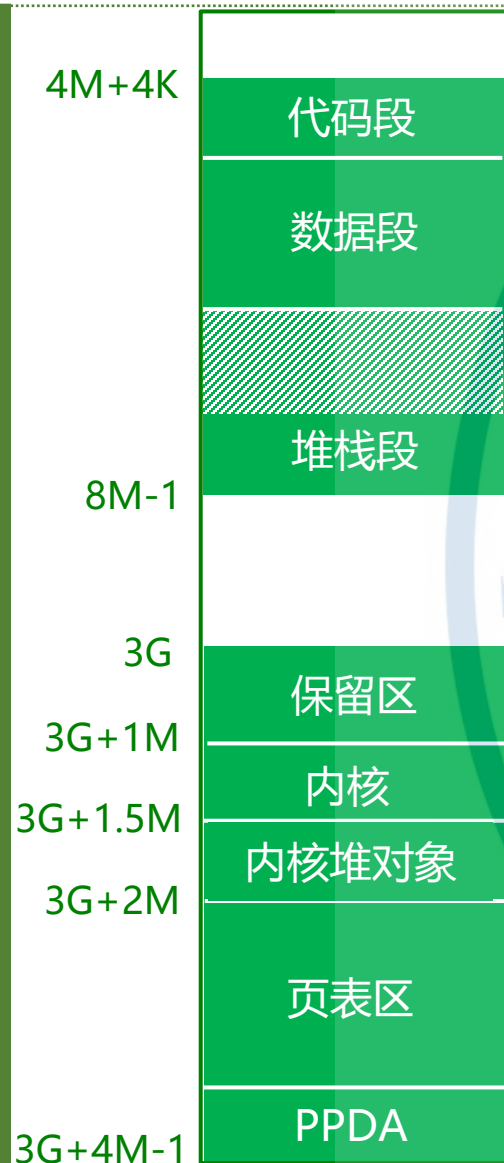
- 程序地址空间
- 物理地址空间
- **地址变换**
- 存储空间管理



UNIX V6++的地址变换



页表的构成

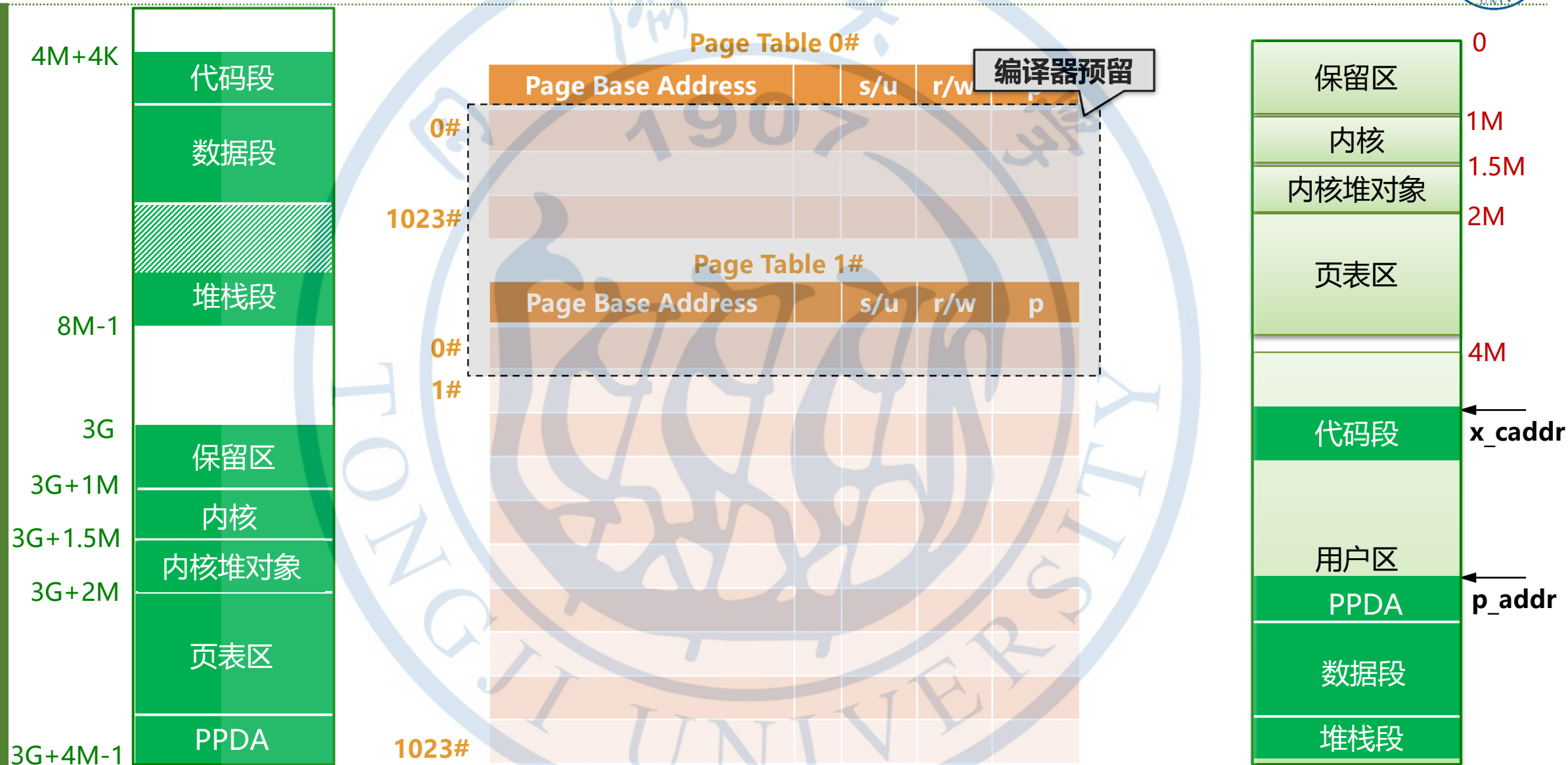




UNIX V6++的地址变换



页表的构成

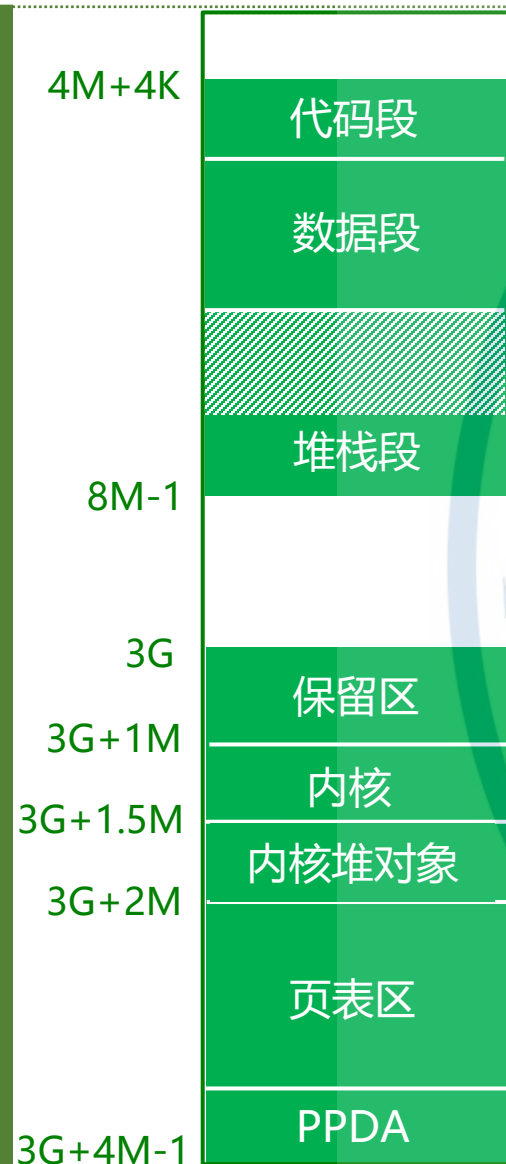




UNIX V6++的地址变换



页表的构成



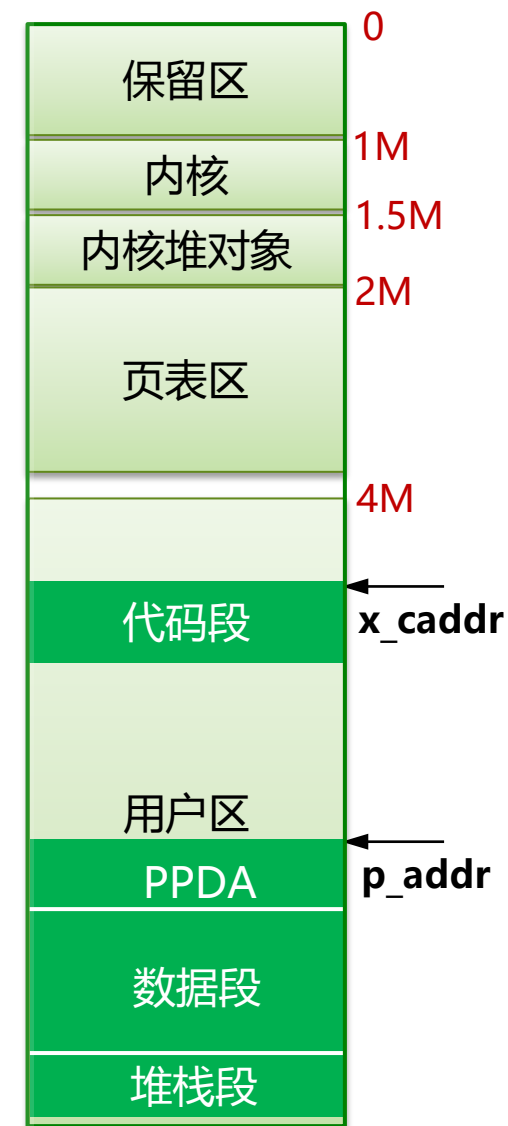
Page Table 0#

	Page Base Address	s/u	r/w	p
0#	/	/	/	/
...	...	...	...	...
1023#	/	/	/	/

编译器预留

Page Table 1#

	Page Base Address	s/u	r/w	p
0#	/	/	/	/
1#				
...				
1023#				

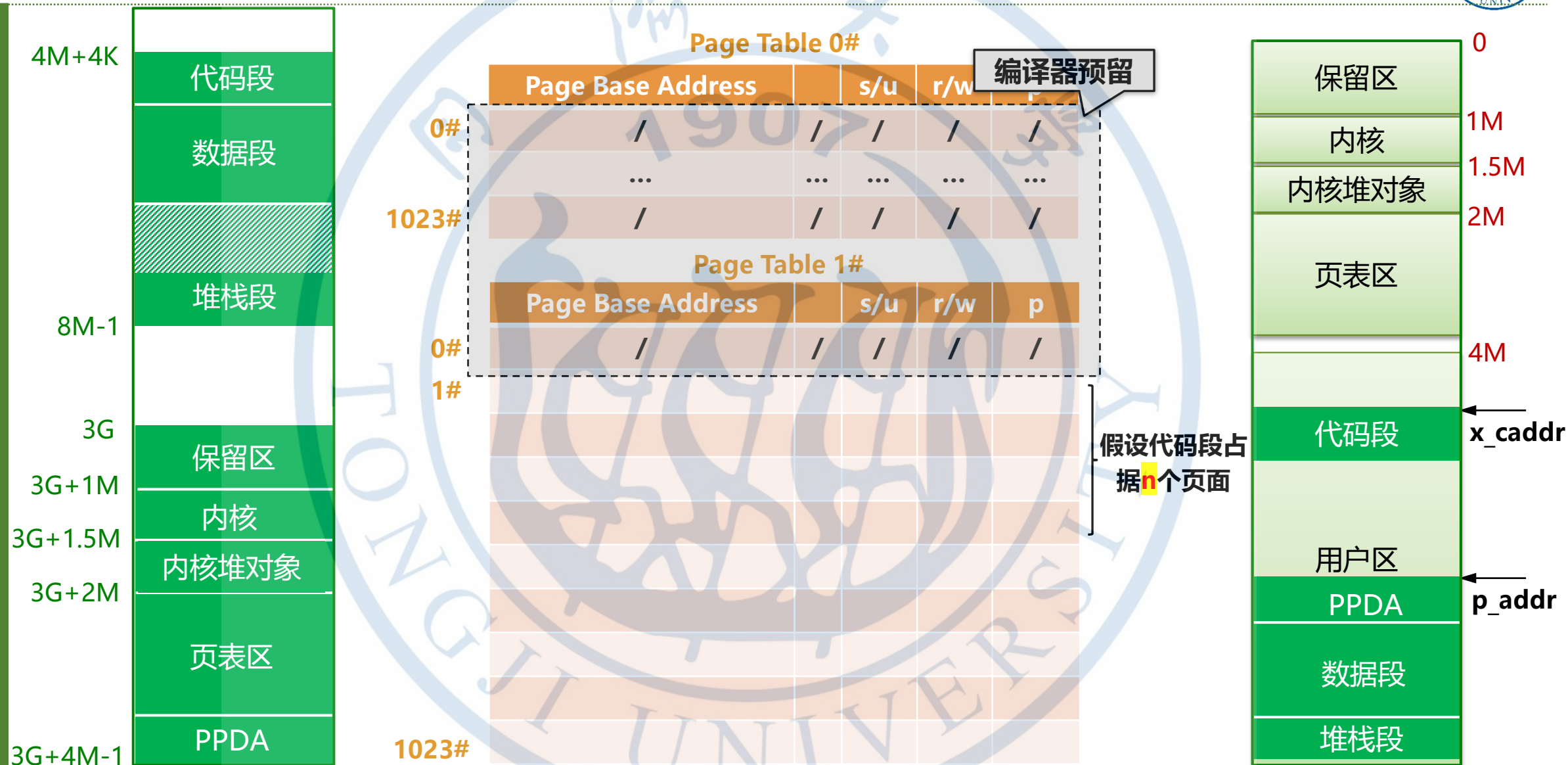




UNIX V6++的地址变换



页表的构成

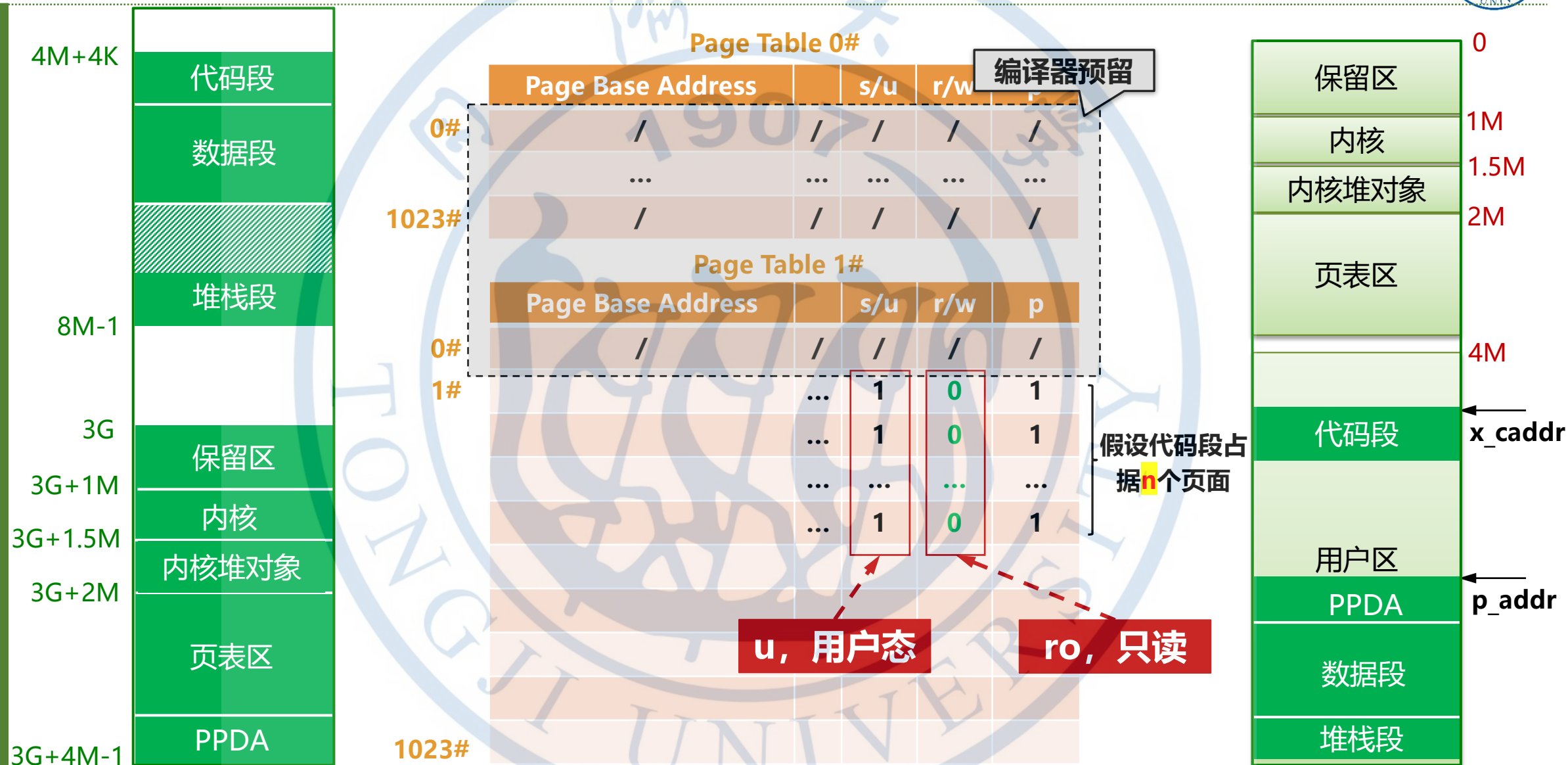




UNIX V6++的地址变换



页表的构成

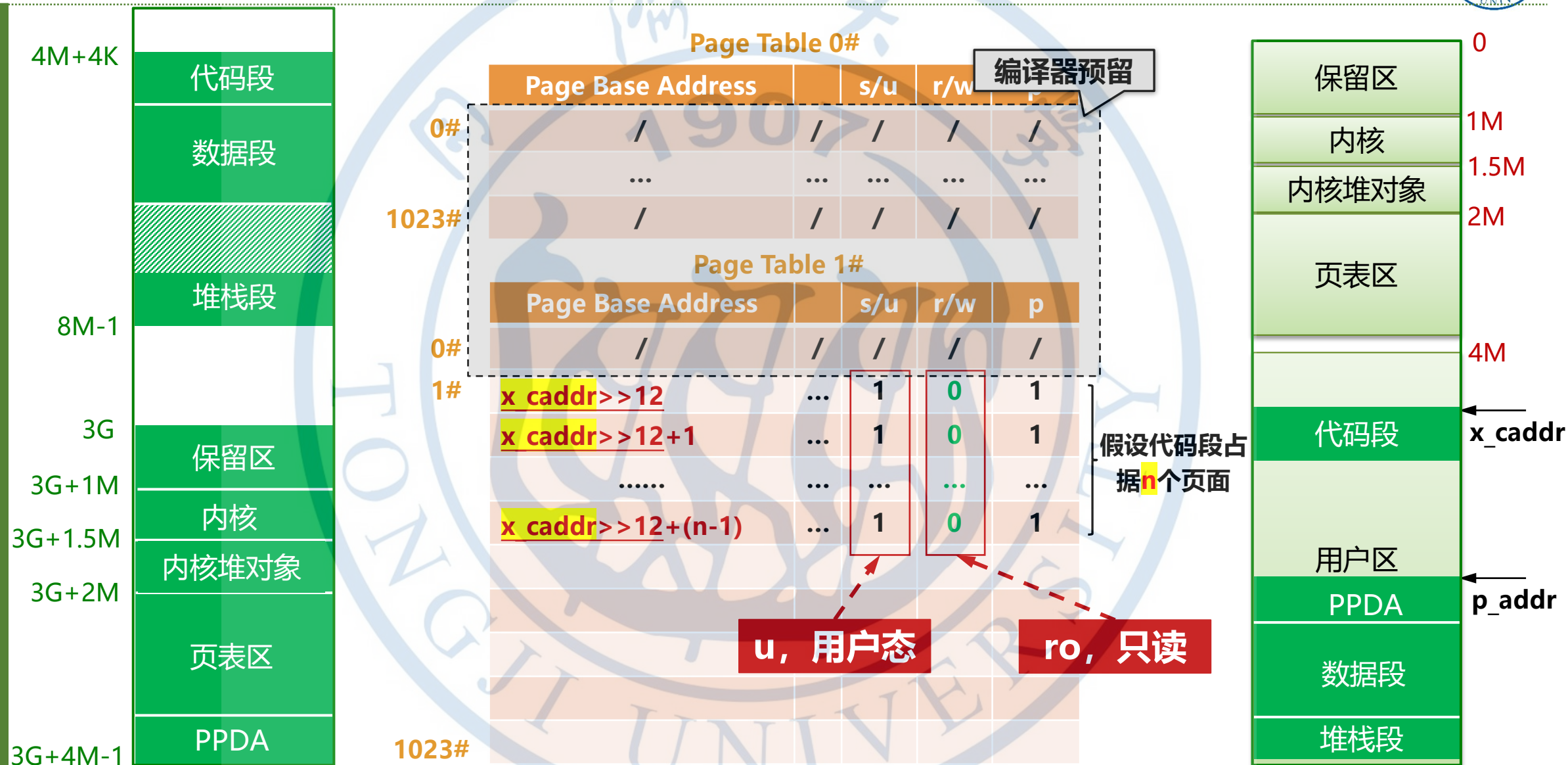




UNIX V6++的地址变换



页表的构成

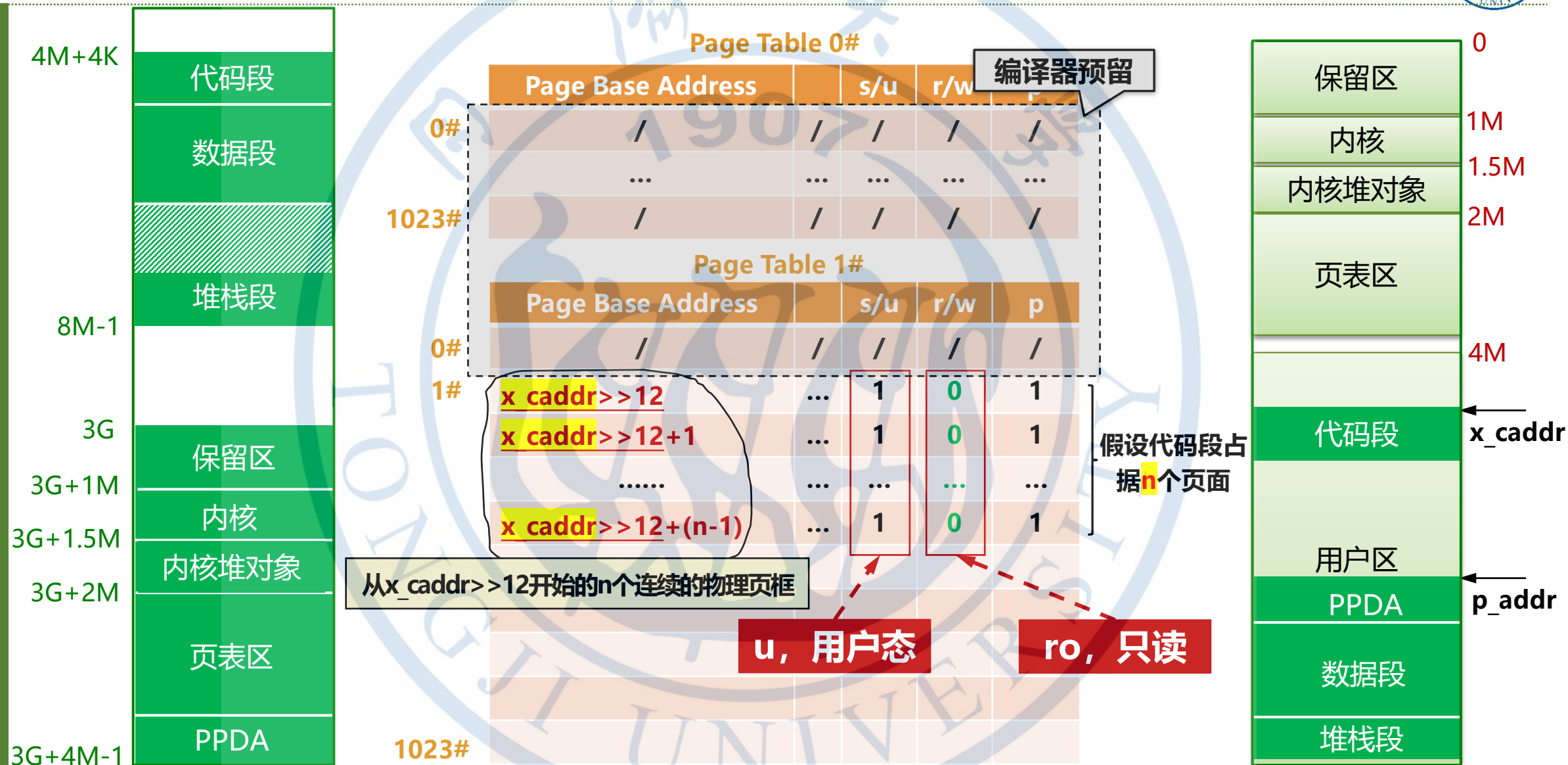




UNIX V6++的地址变换



页表的构成

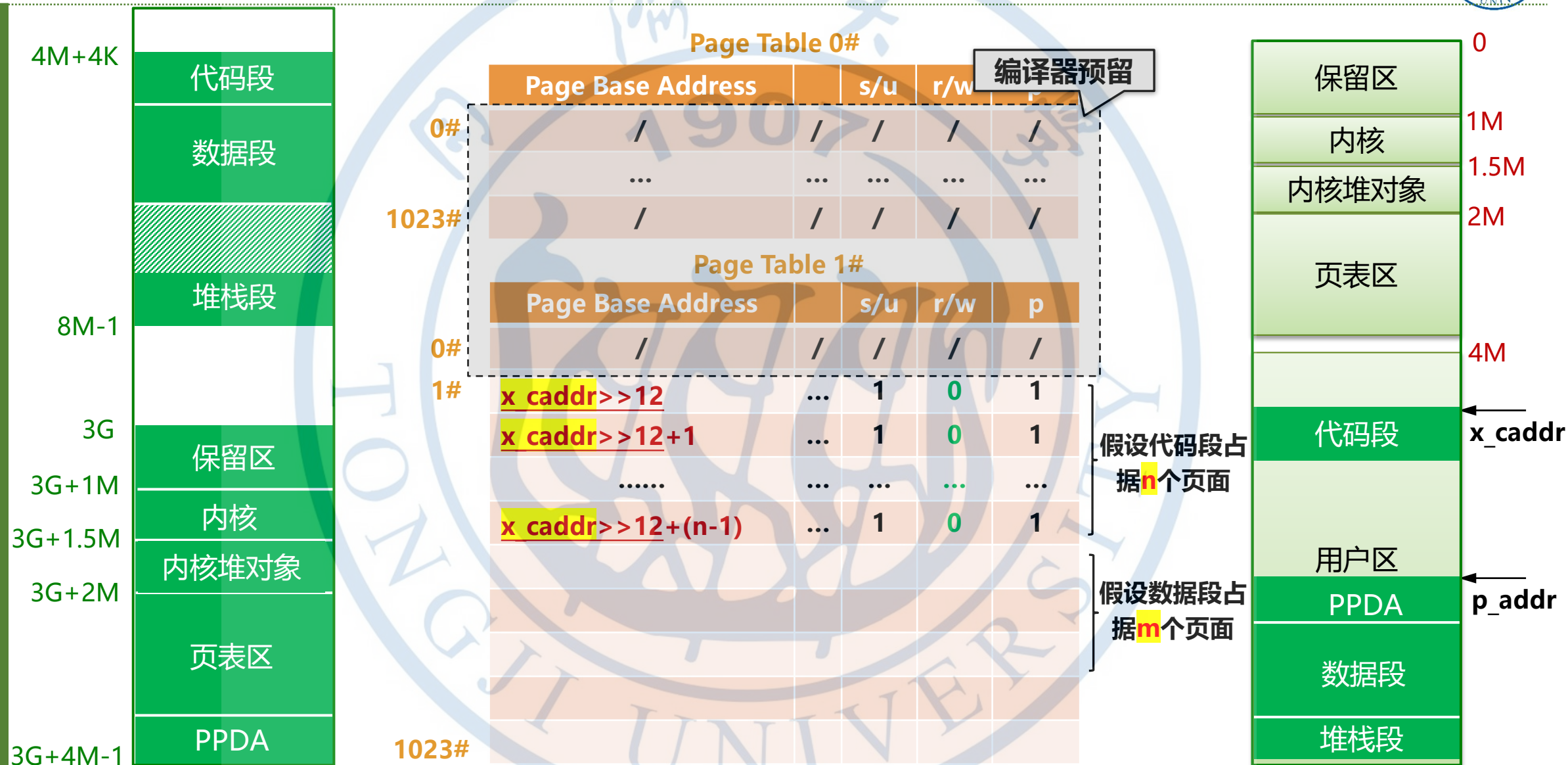




UNIX V6++的地址变换



页表的构成





UNIX V6++的地址变换



页表的构成

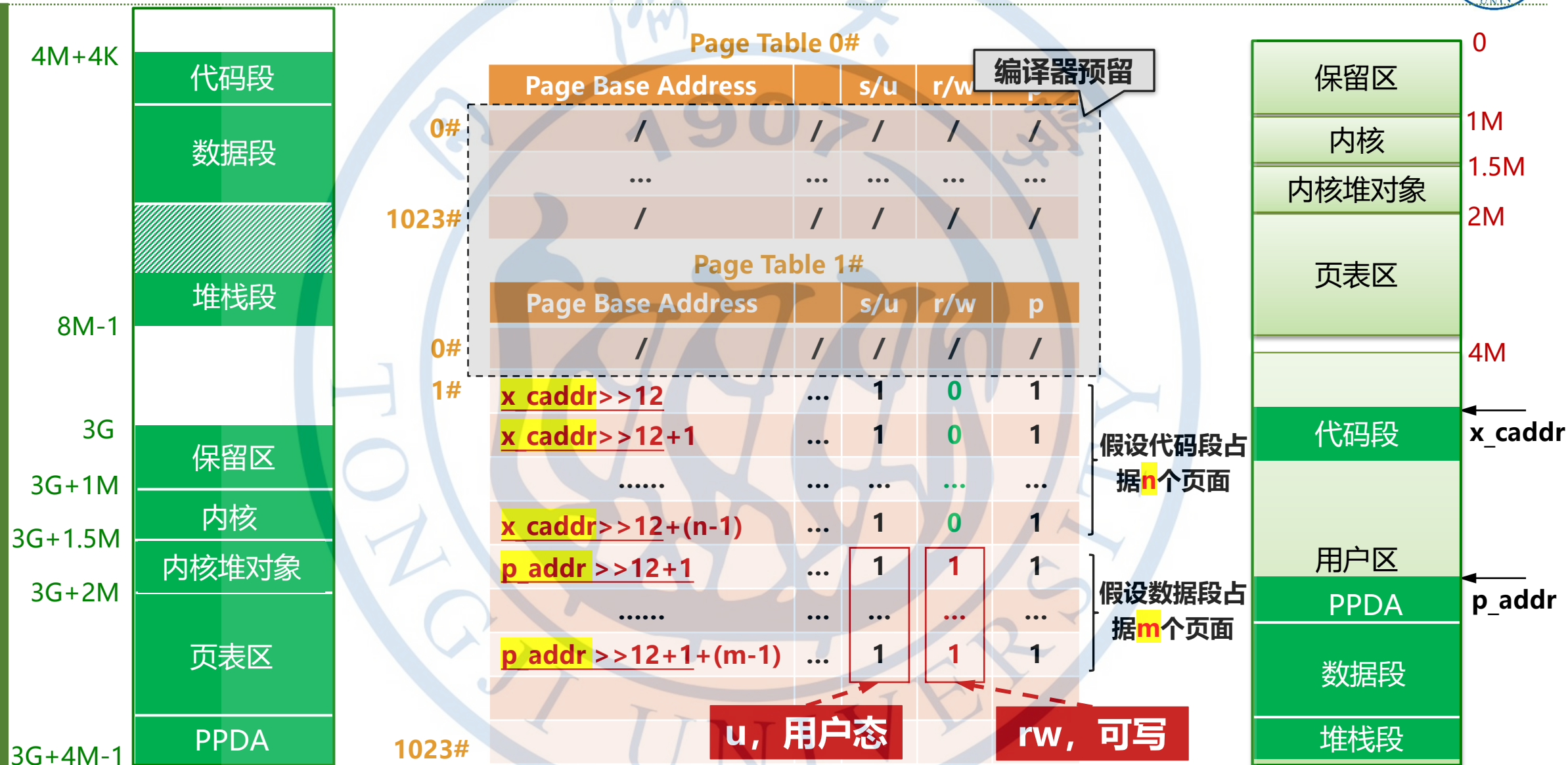




UNIX V6++的地址变换



页表的构成

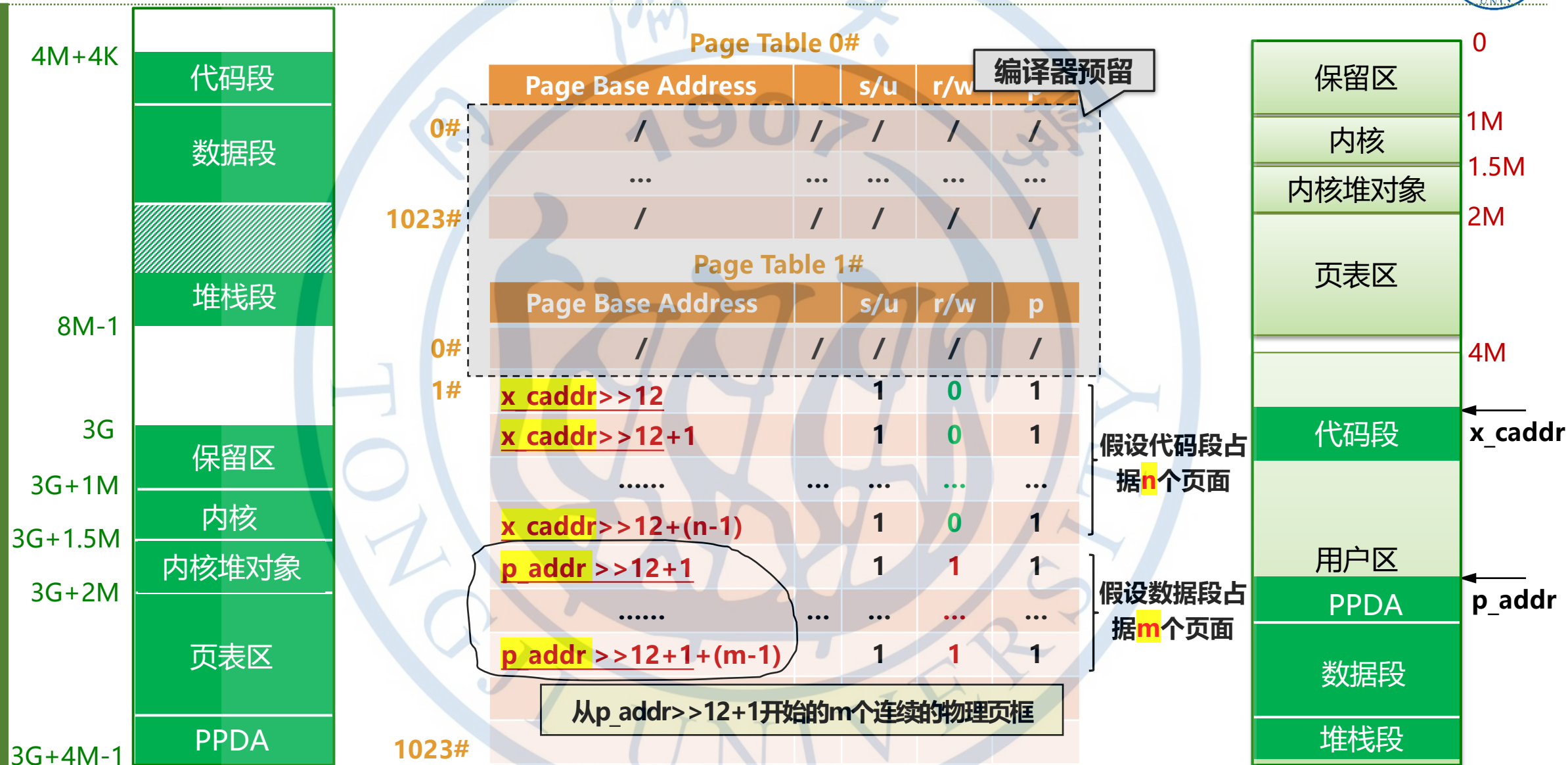




UNIX V6++的地址变换



页表的构成

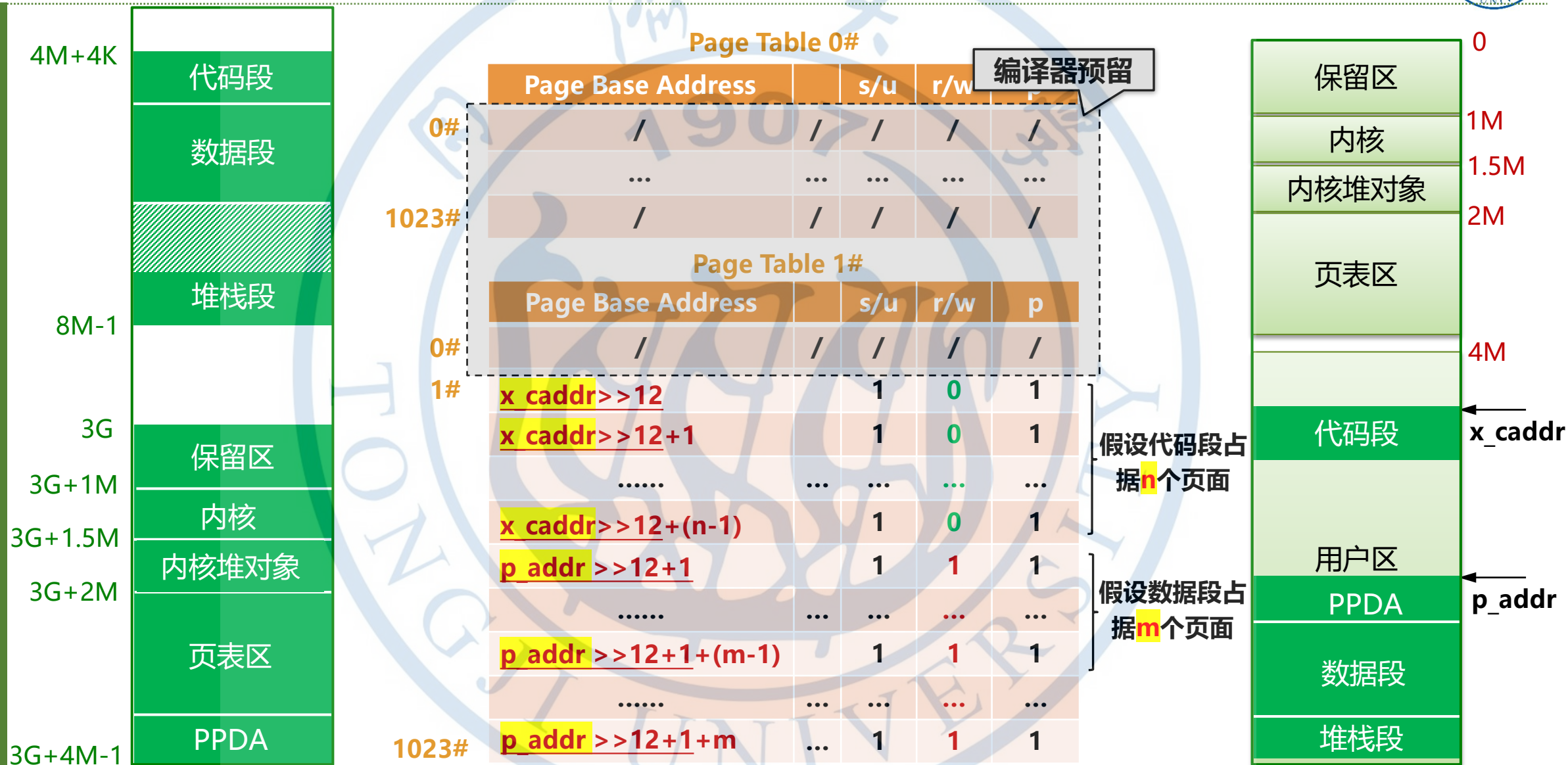




UNIX V6++的地址变换



页表的构成

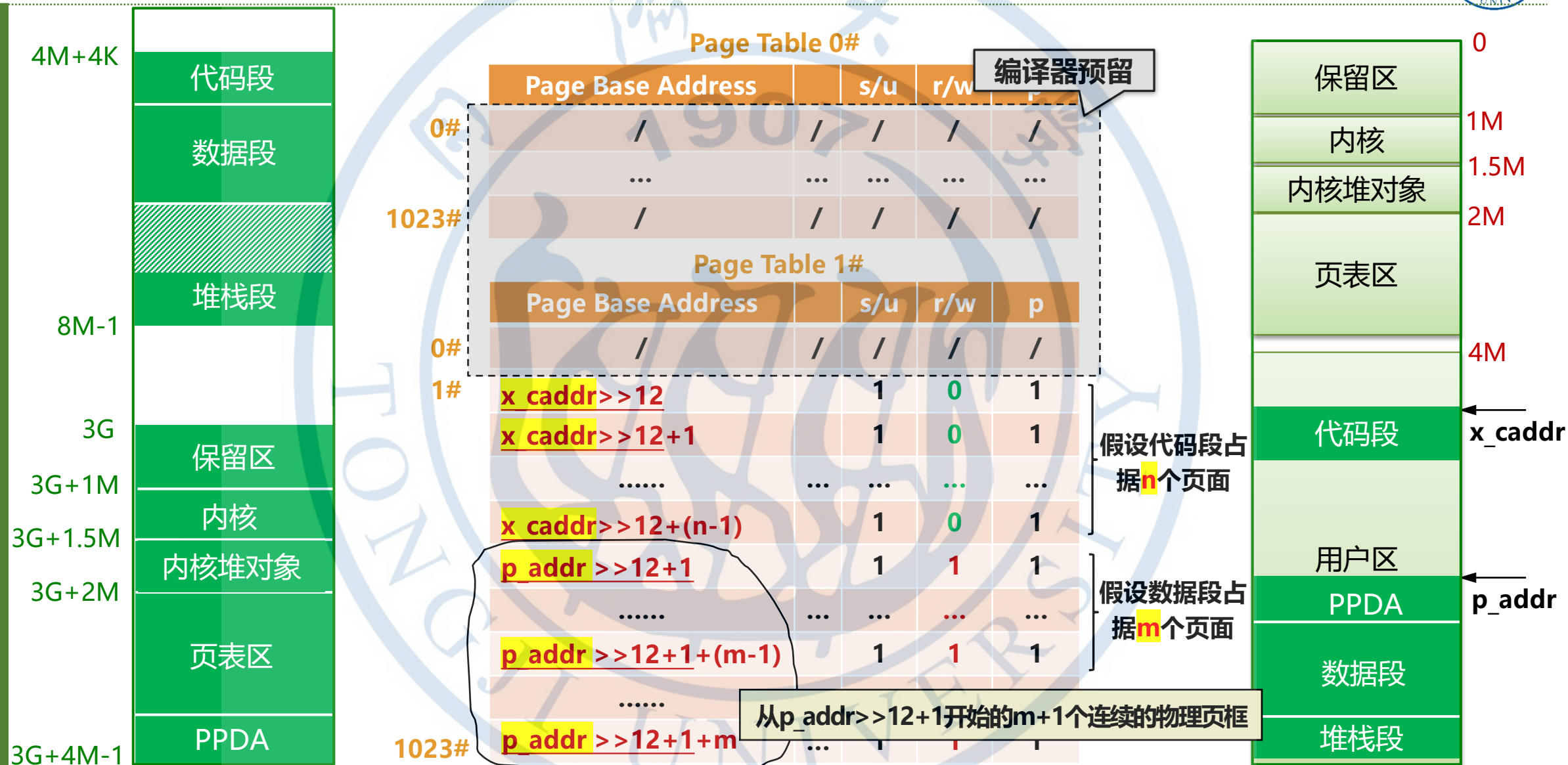




UNIX V6++的地址变换



页表的构成

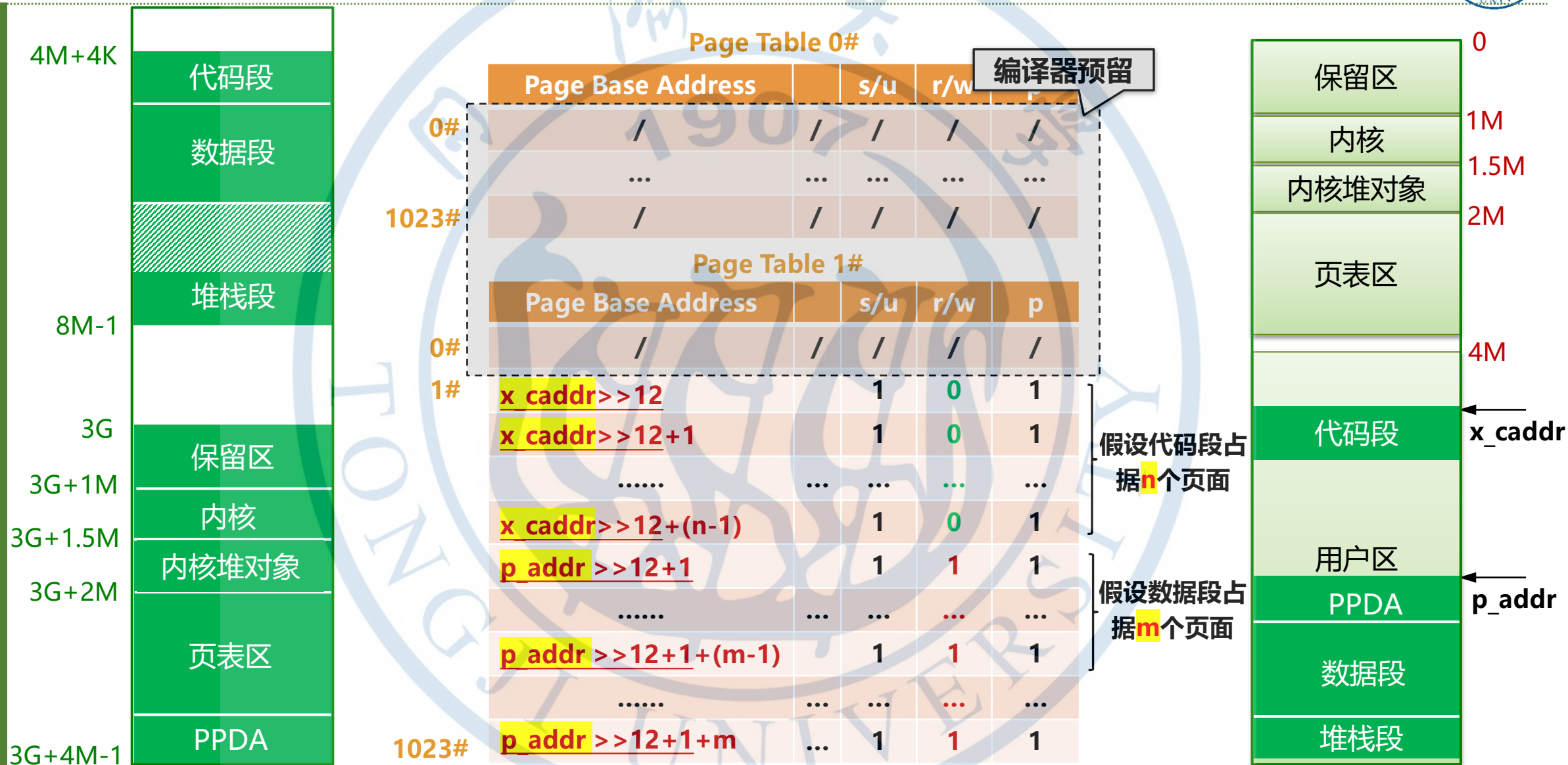




UNIX V6++的地址变换



页表的构成

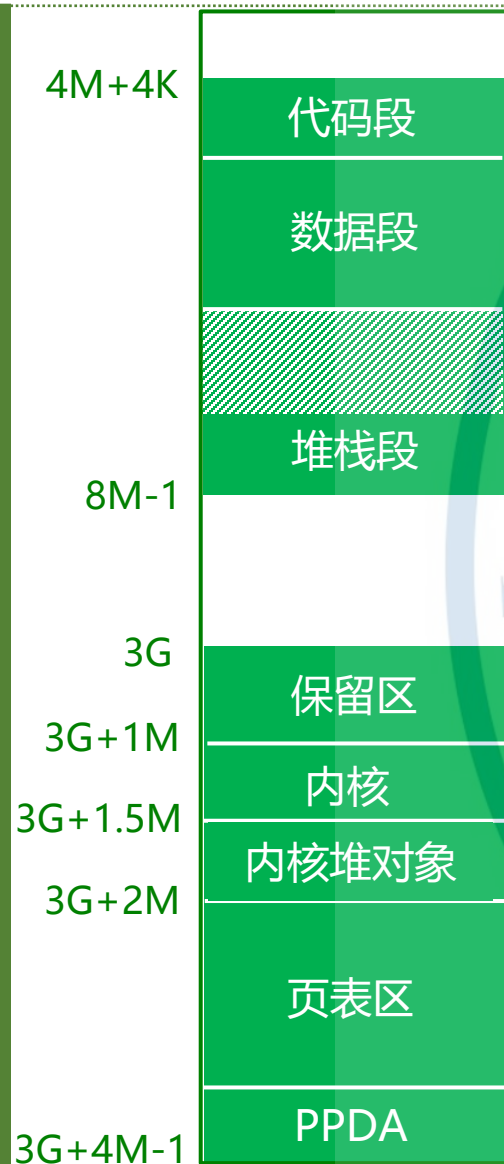




UNIX V6++的地址变换

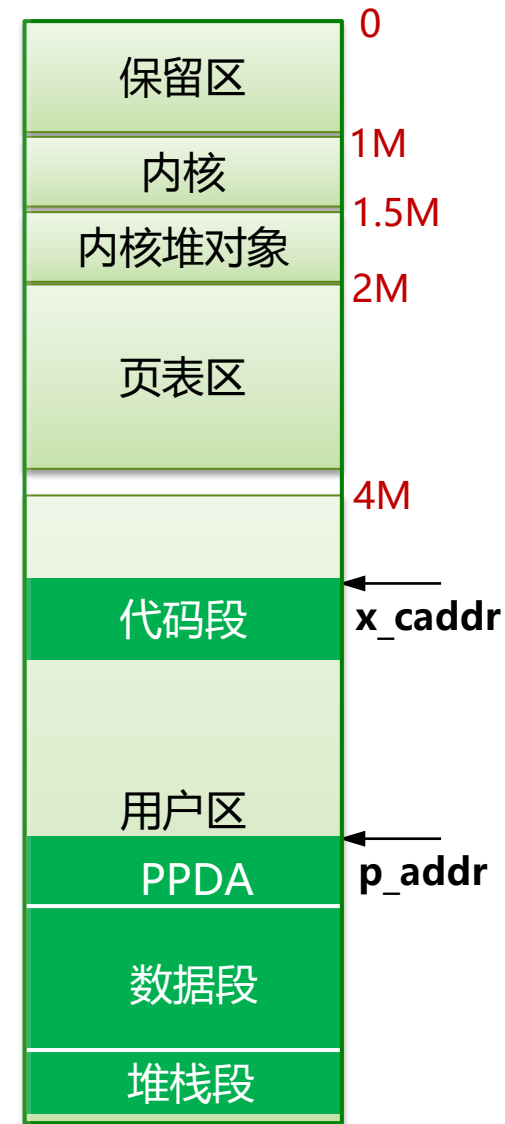


页表的构成



Page Table 768#

Page Base Address	s/u	r/w	p
0#			
1#			
1022#			
1023#			

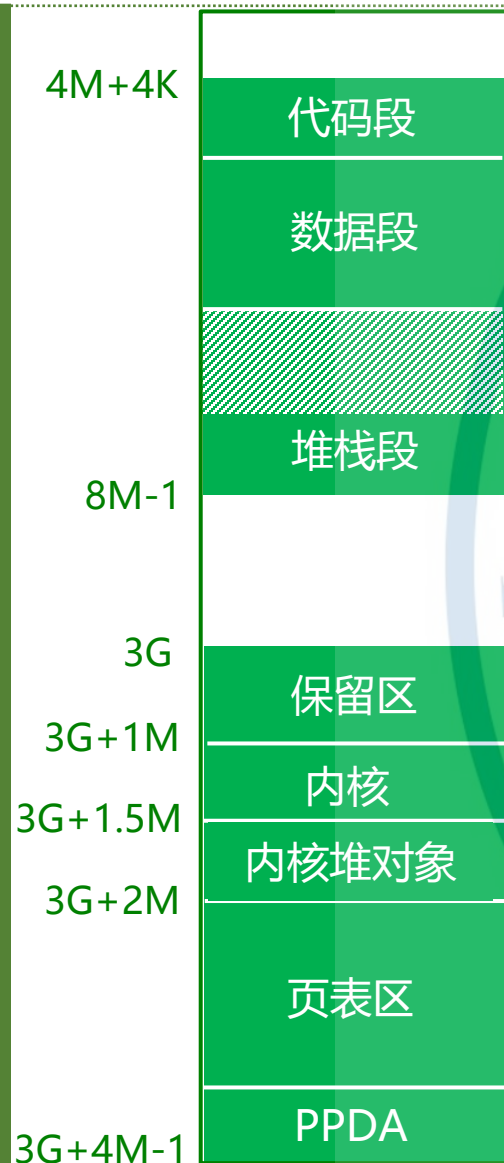




UNIX V6++的地址变换



页表的构成

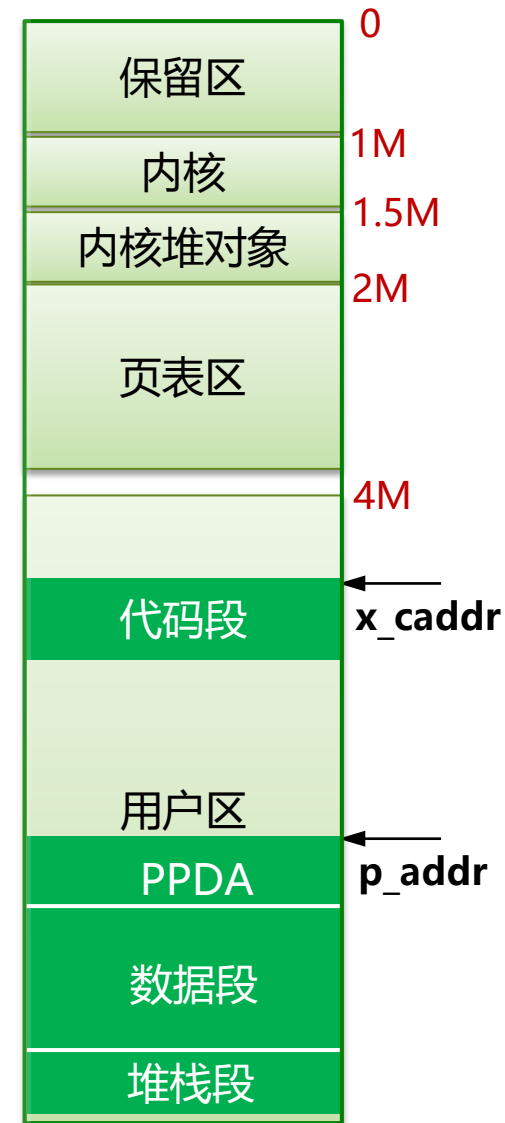


Page Table 768#

Page Base Address		s/u	r/w	p
0	...	0	1	1
1	...	0	1	1
2	...	0	1	1
.....	...	...	...	...
1022#	1022	0	1	1
1023#				

s, 内核态

rw, 读写

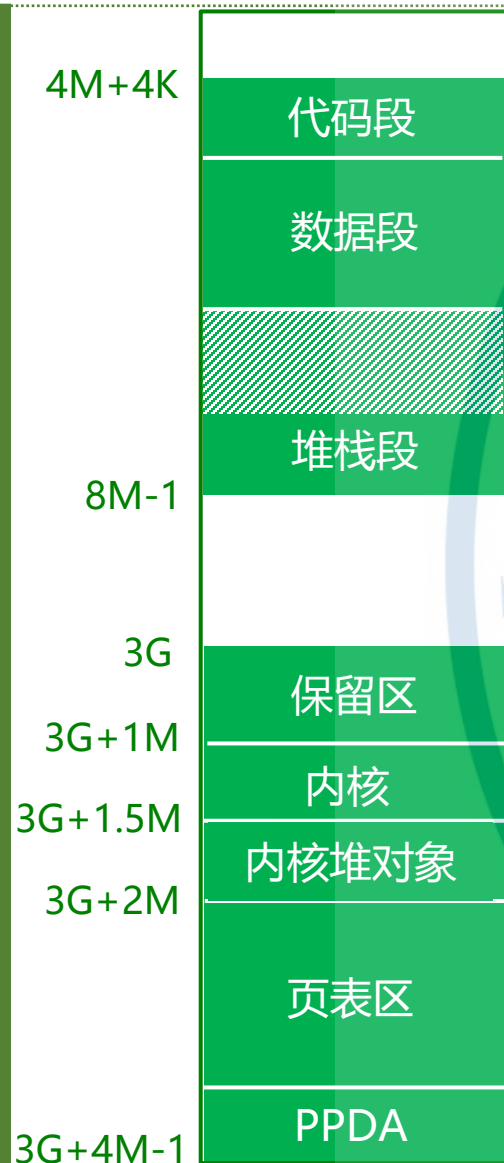




UNIX V6++的地址变换

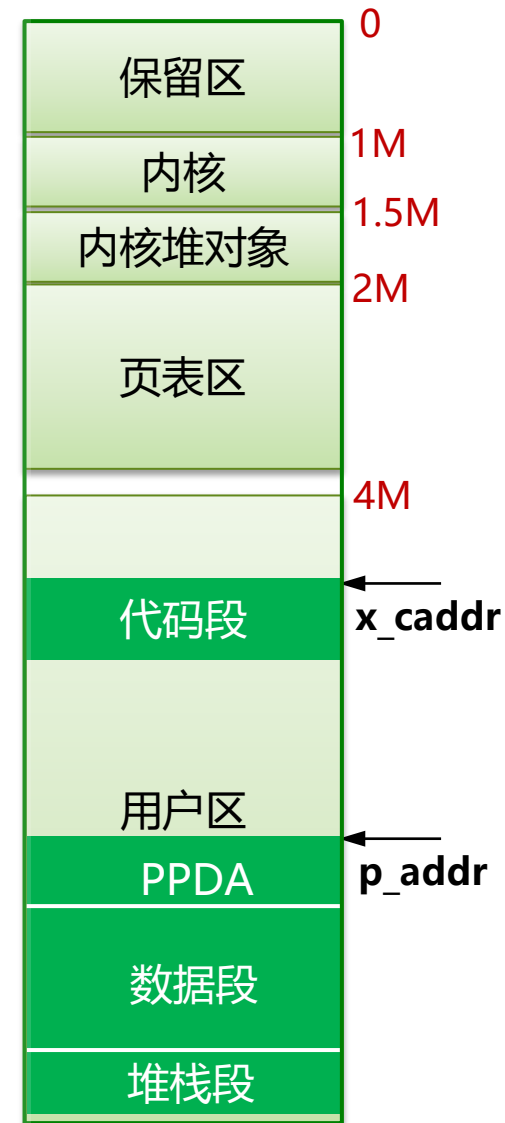


页表的构成



Page Table 768#

Page Base Address		s/u	r/w	p
0	...	0	1	1
1	...	0	1	1
2	...	0	1	1
.....	...	...	...	...
1022	...	0	1	1
p_addr >> 12	...	0	1	1

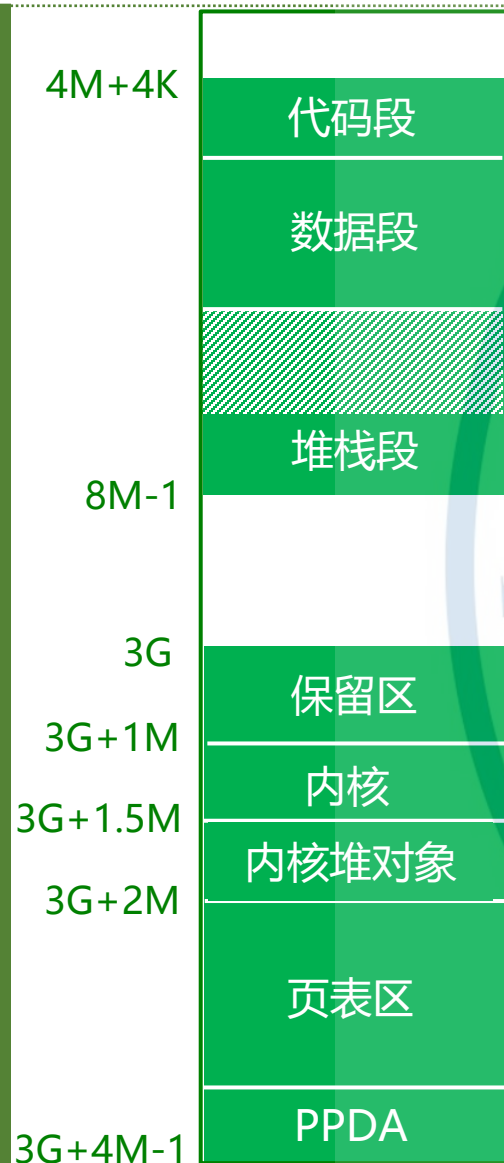




UNIX V6++的地址变换



页表的构成



Page Table 0#

	Page Base Address		s/u	r/w	p
0#	/	/	/	/	/
	...	...	...	...	...
1023#	/	/	/	/	/

Page Table 1#

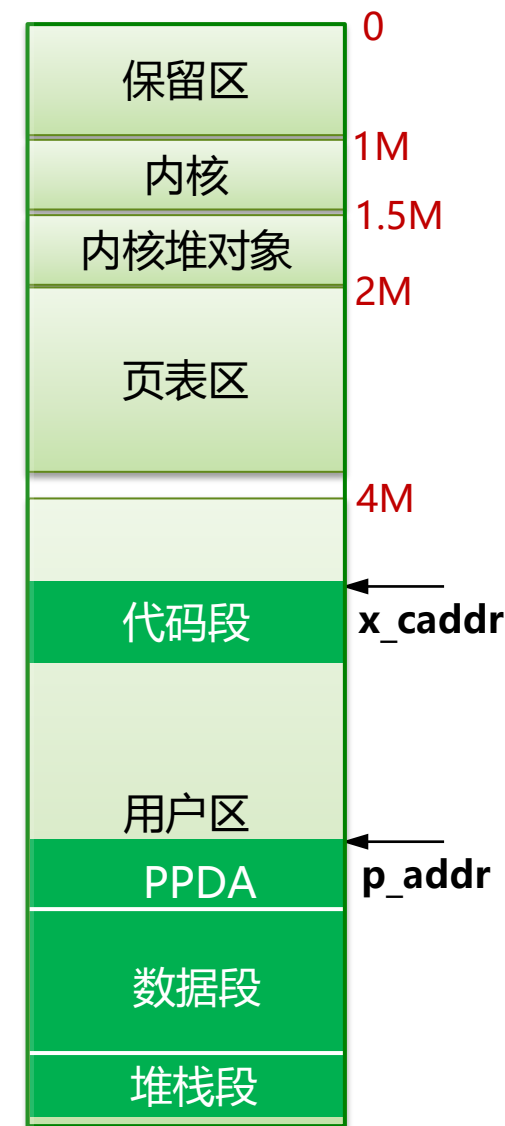
	Page Base Address		s/u	r/w	p
0#	/	/	/	/	/
1#	$x_caddr >> 12$	1	0	1	1
	$x_caddr >> 12+1$	1	0	1	1
		...	...	...	...
	$x_caddr >> 12+(n-1)$	1	0	1	1
	$p_addr >> 12+1$	1	1	1	1
		...	...	...	...
	$p_addr >> 12+1+(m-1)$	1	1	1	1
		...	...	...	...
1023#	$p_addr >> 12+1+m$	...	1	1	1

Page Table 768#

	Page Base Address		s/u	r/w	p
0#	0	...	0	1	1
1#	1	...	0	1	1
	2	...	0	1	1
		...	...	...	...
1022#	1022	...	0	1	1
1023#	$p_addr >> 12$	...	0	1	1

三张页表保存在哪里?

页目录保存在哪里?

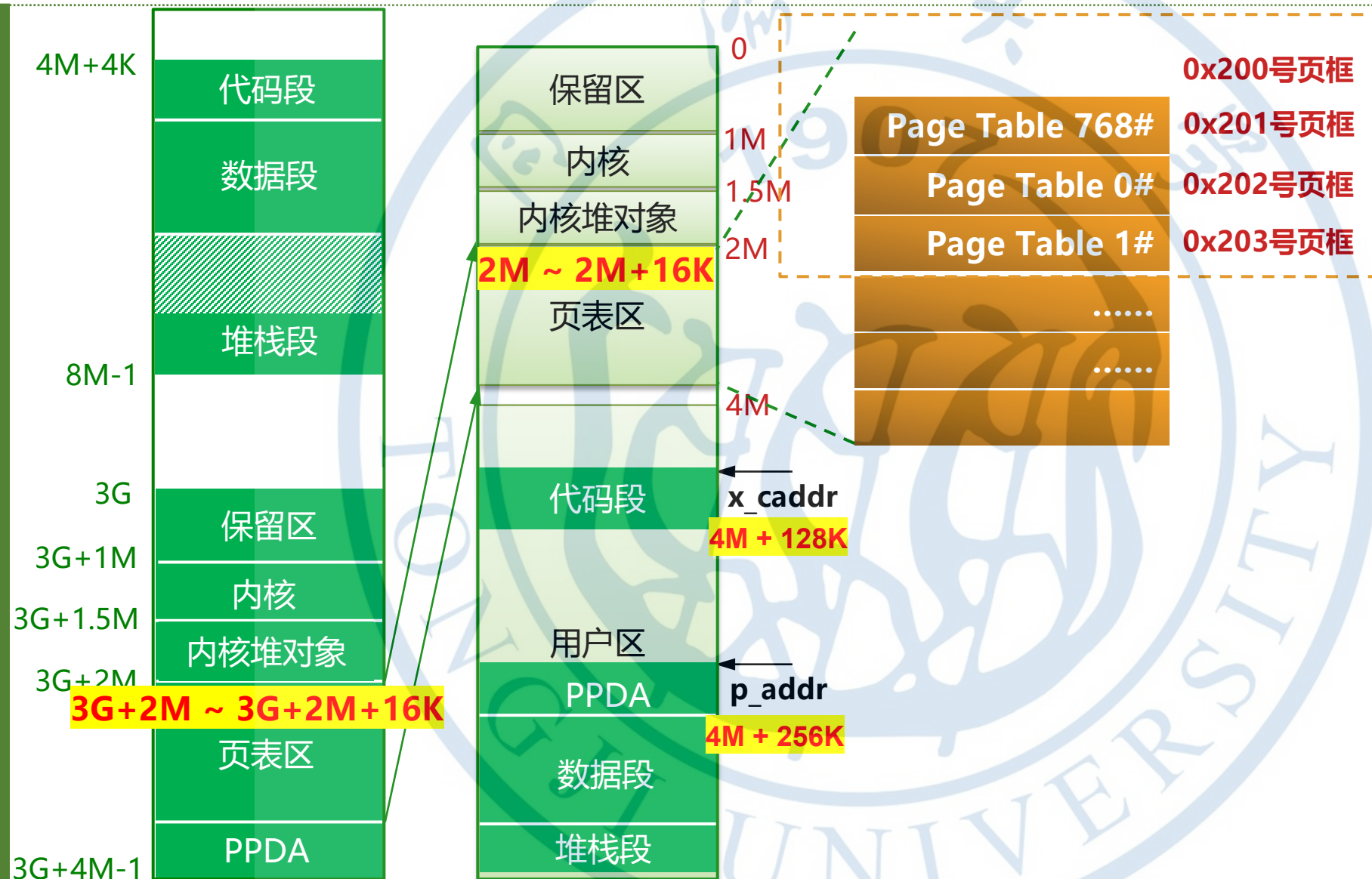




UNIX V6++的地址变换



现运行进程的页表



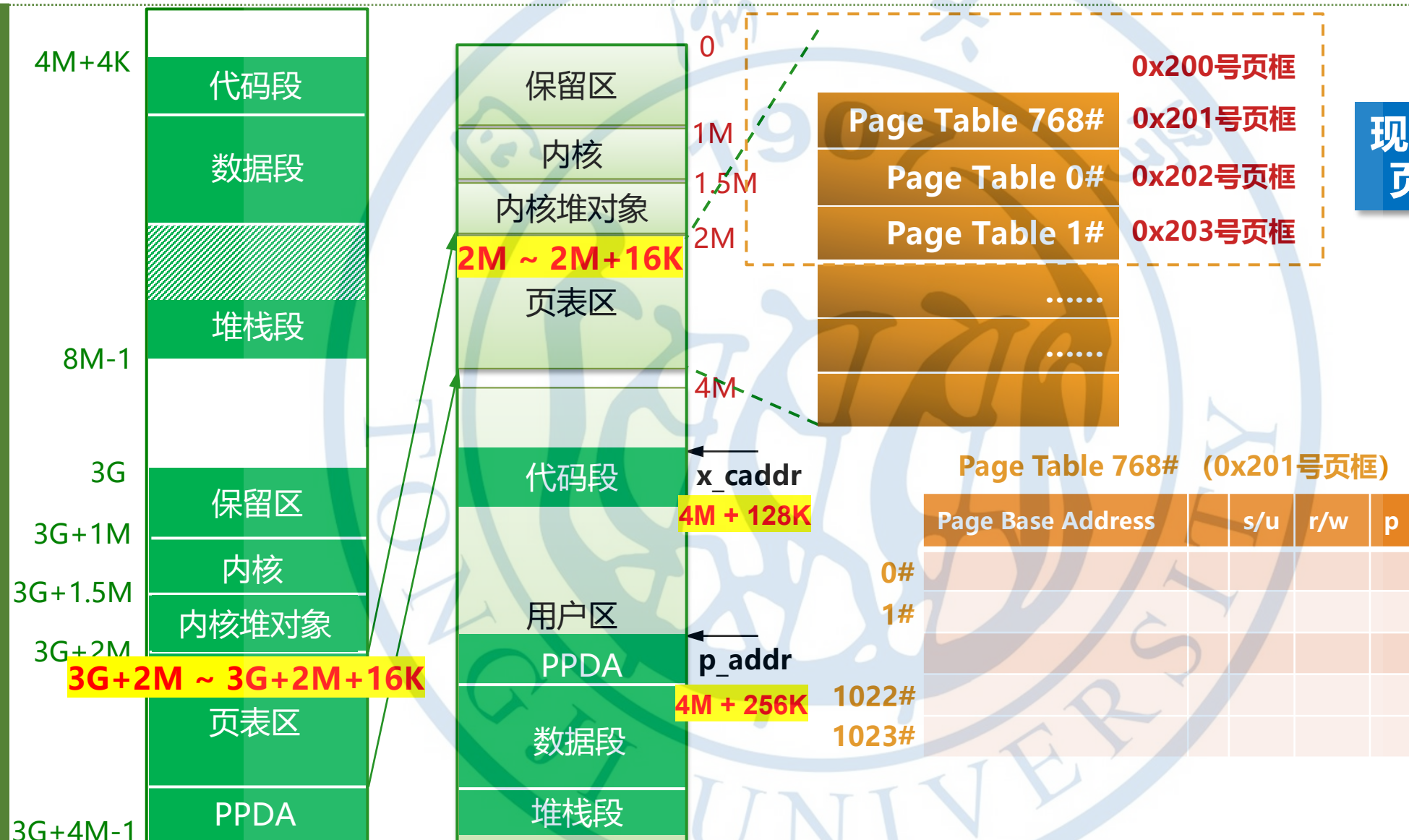
现运行进程的内核
页表与用户页表



UNIX V6++的地址变换



现运行进程的页表



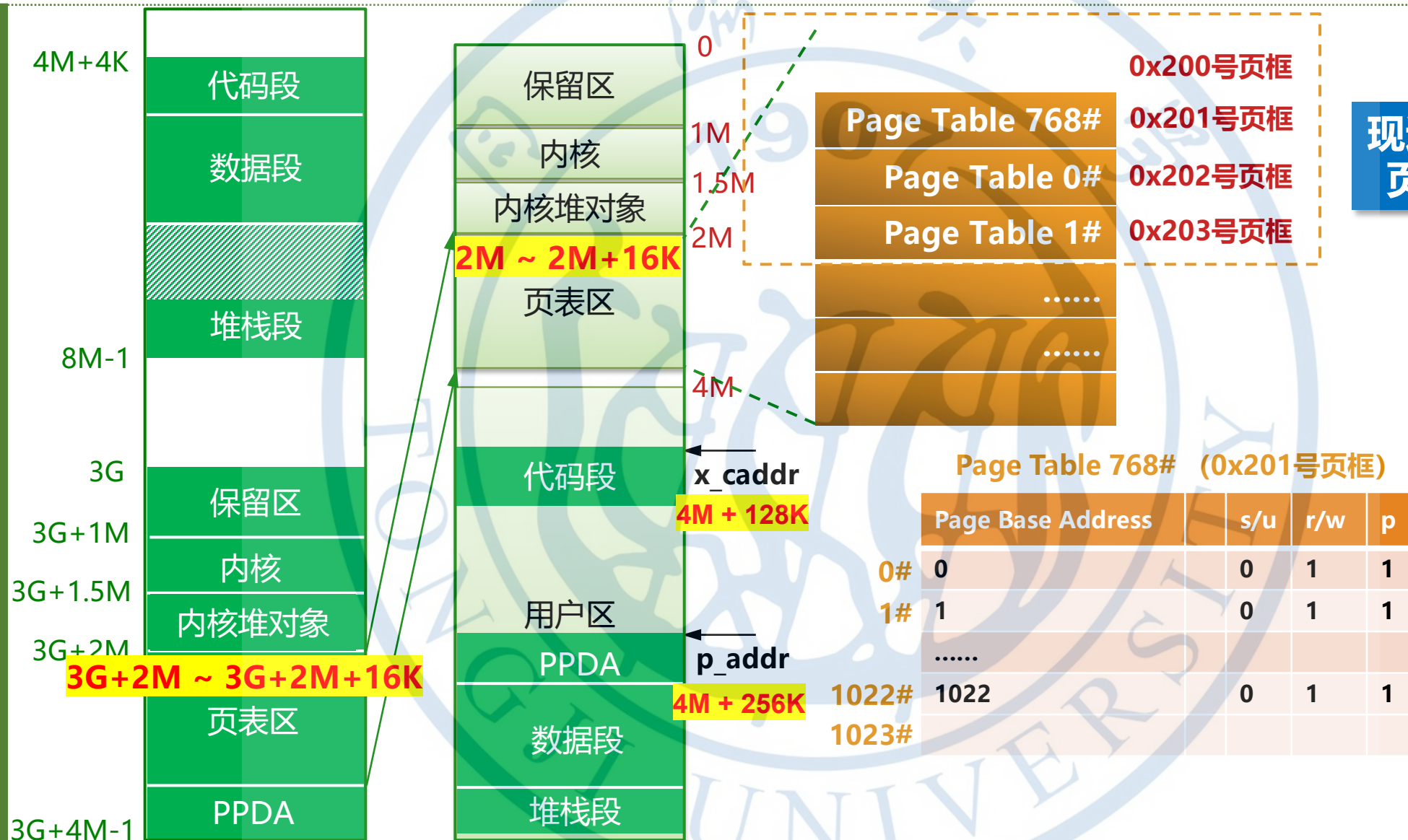
现运行进程的内核
页表与用户页表



UNIX V6++的地址变换



现运行进程的页表



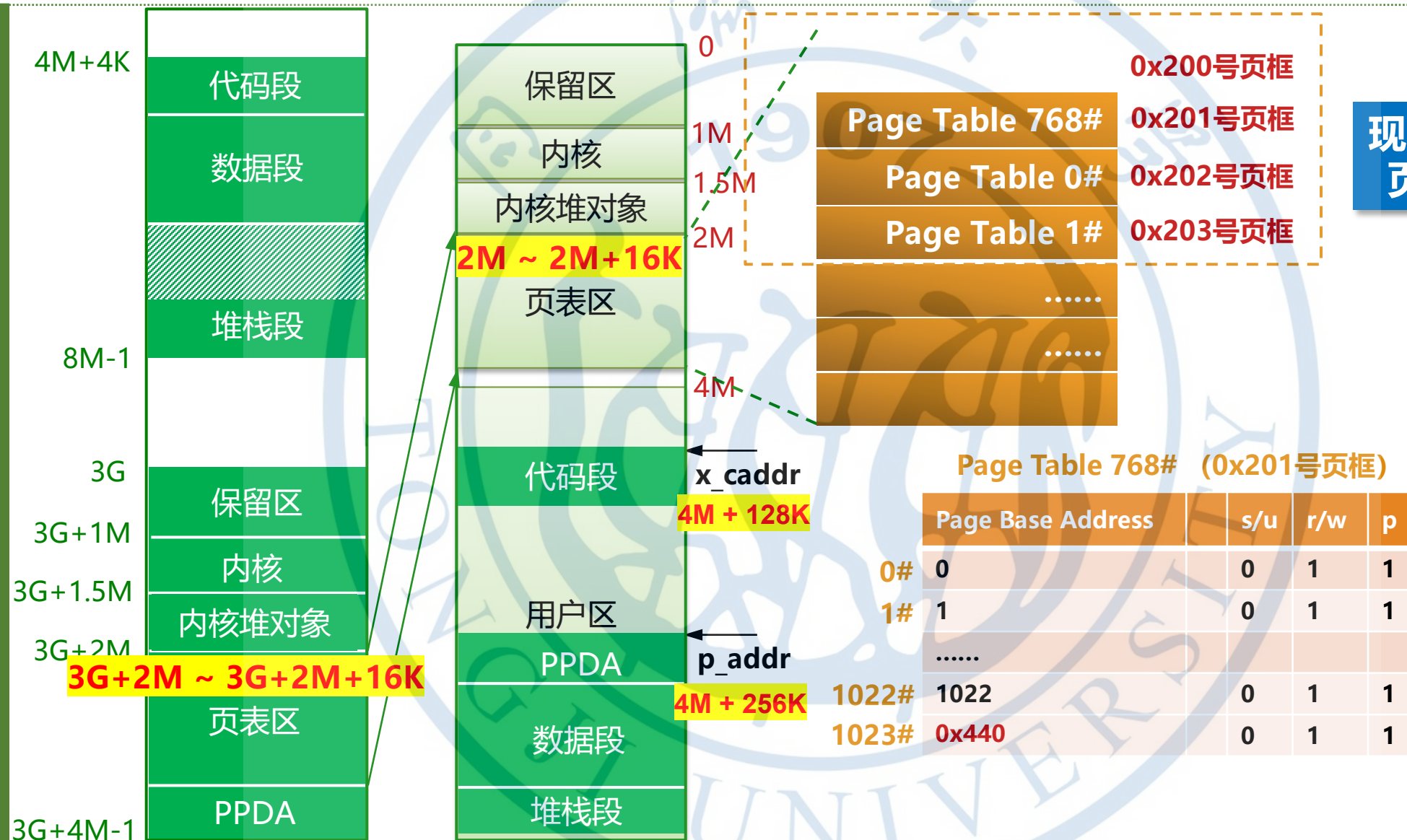
现运行进程的内核
页表与用户页表



UNIX V6++的地址变换



现运行进程的页表



现运行进程的内核
页表与用户页表

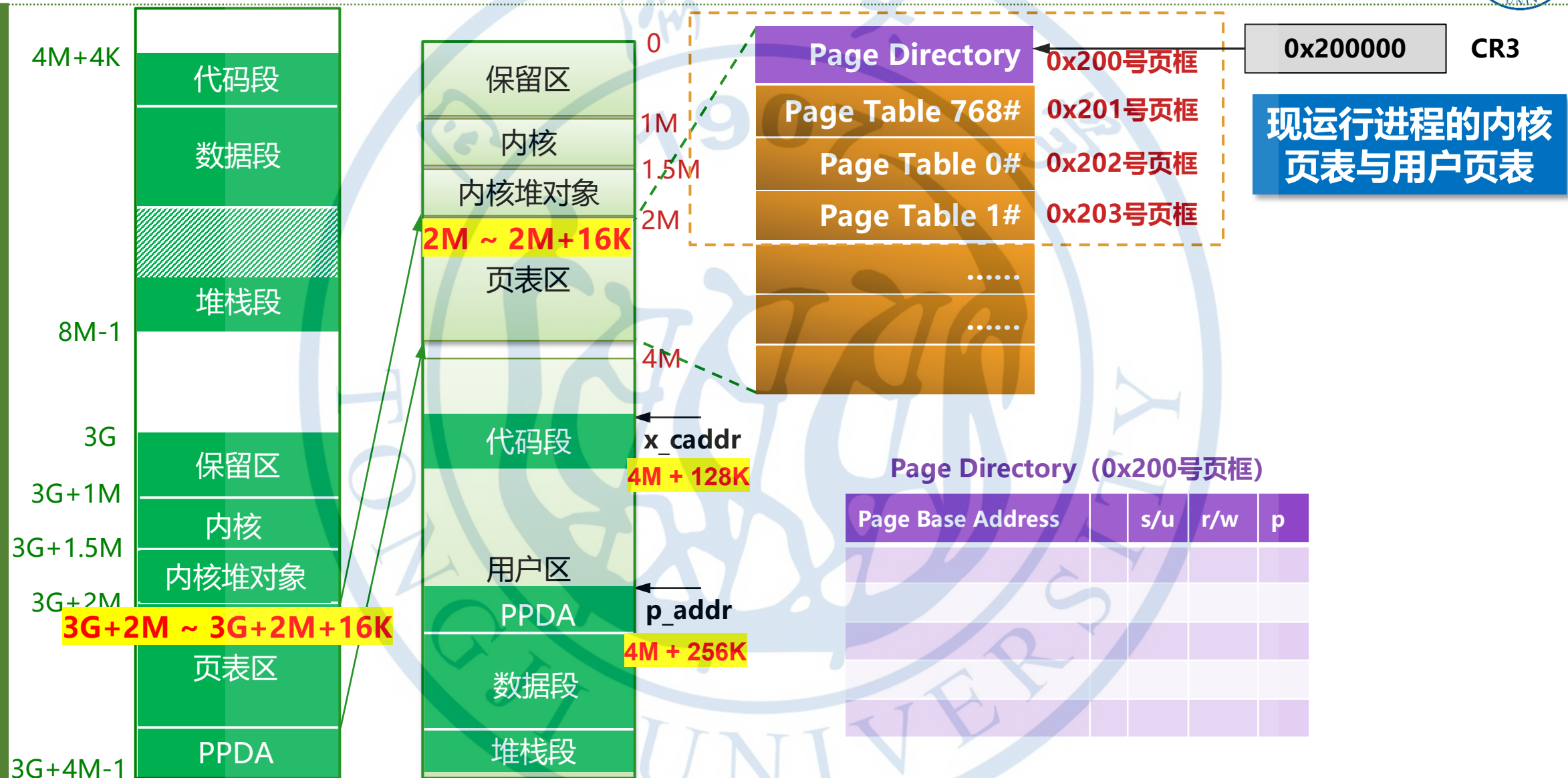
2024-2025-1, Fang Yu



UNIX V6++的地址变换



现运行进程的页表

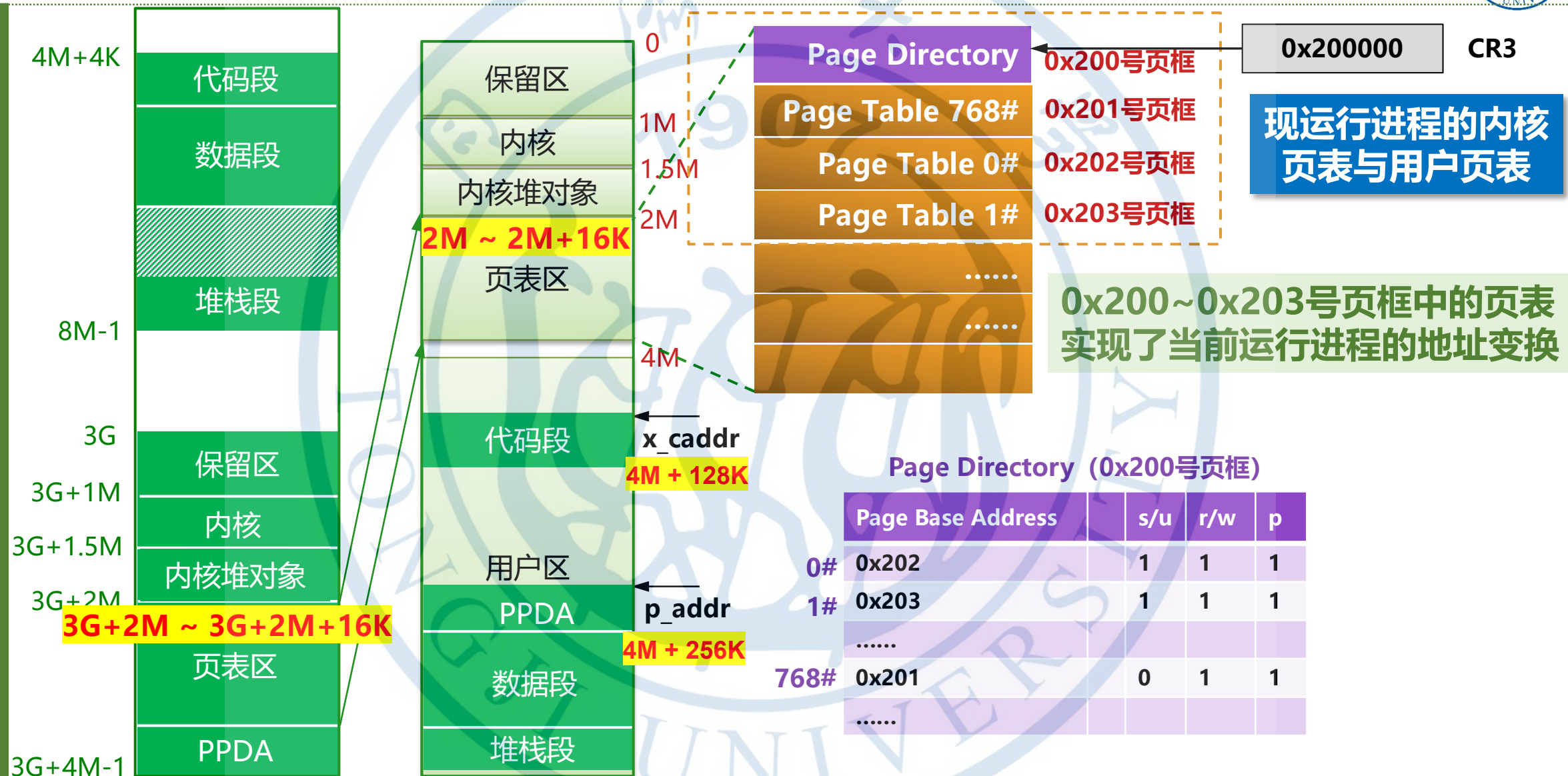




UNIX V6++的地址变换



现运行进程的页表

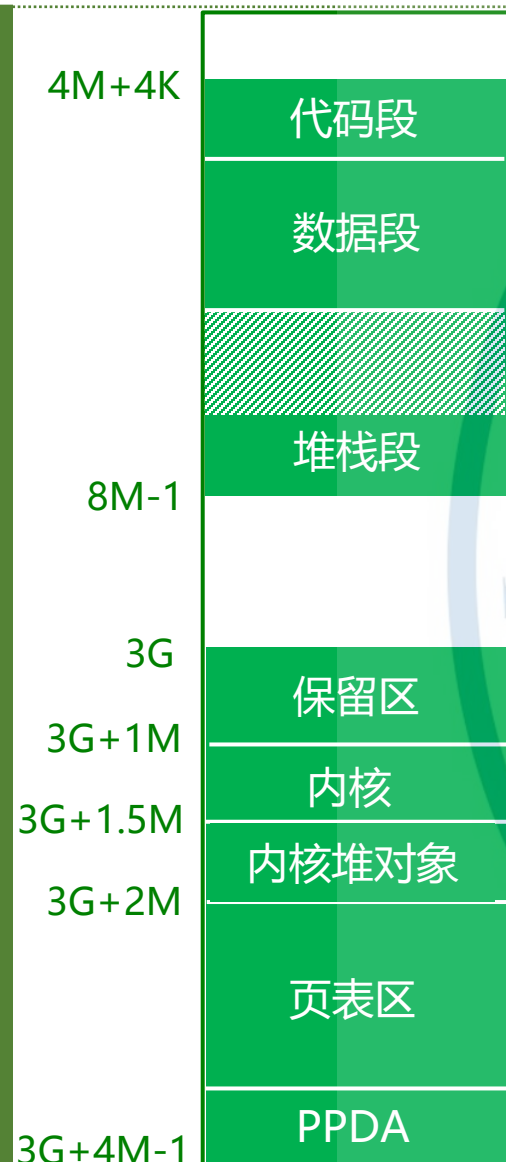




UNIX V6++的地址变换



相关的数据结构定义



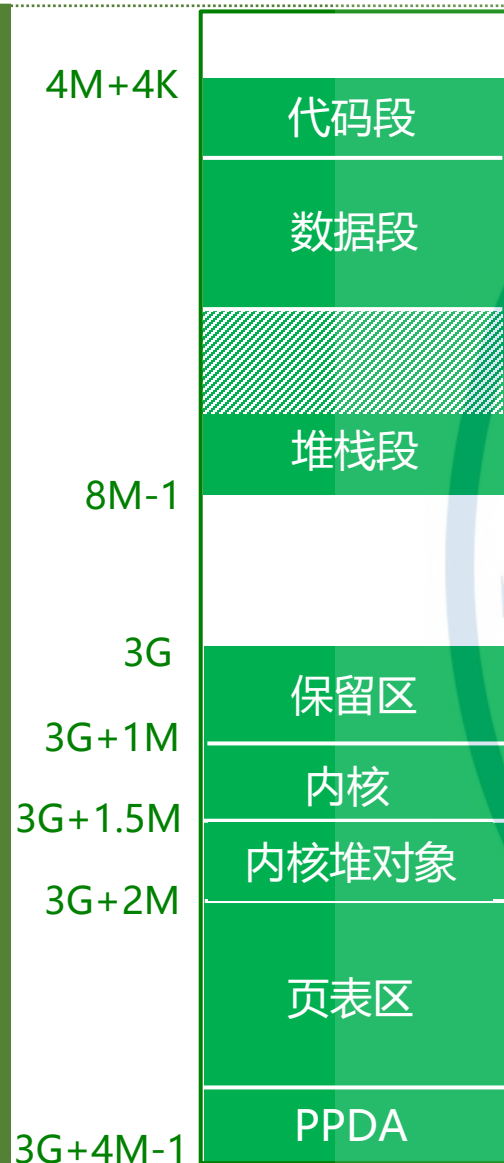
```
class Machine
{
public:
    .....;
    /* 页目录、核心态页表、用户态页表在物理内存中的起始地址 */
    static const unsigned long PAGE_DIRECTORY_BASE_ADDRESS = 0x200000;
    static const unsigned long KERNEL_PAGE_TABLE_BASE_ADDRESS = 0x201000;
    static const unsigned long USER_PAGE_TABLE_BASE_ADDRESS = 0x202000;
    static const unsigned long USER_PAGE_TABLE_CNT = 2;
    /* 内核空间大小 4M 0xC0000000 - 0xC0400000 1 PageTable */
    static const unsigned int KERNEL_SPACE_SIZE = 0x400000;
    static const unsigned long KERNEL_SPACE_START_ADDRESS = 0xC0000000;
public:
    void InitPageDirectory();
    void InitUserPageTable();
    PageDirectory& GetPageDirectory(); /* 获取当前正在使用的页目录表 */
    PageTable& GetKernelPageTable(); /* 获取操作系统内核所使用的页表, */
    PageTable* GetUserPageTableArray(); /* 获取用户进程页表, 共两张 */
    .....;
private:
    PageDirectory* m_PageDirectory; /* 指向系统页目录 */
    PageTable* m_KernelPageTable; /* 指向内核页表 */
    PageTable* m_UserPageTable; /* 指向用户页表 */
    .....;
};
```



UNIX V6++的地址变换



页目录和核心页表的初始化



```
void Machine::InitPageDirectory()  
{  
    PageDirectory* pPageDirectory = (PageDirectory*)  
        (PAGE_DIRECTORY_BASE_ADDRESS + KERNEL_SPACE_START_ADDRESS);  
    .....  
}
```

0x200000

3G

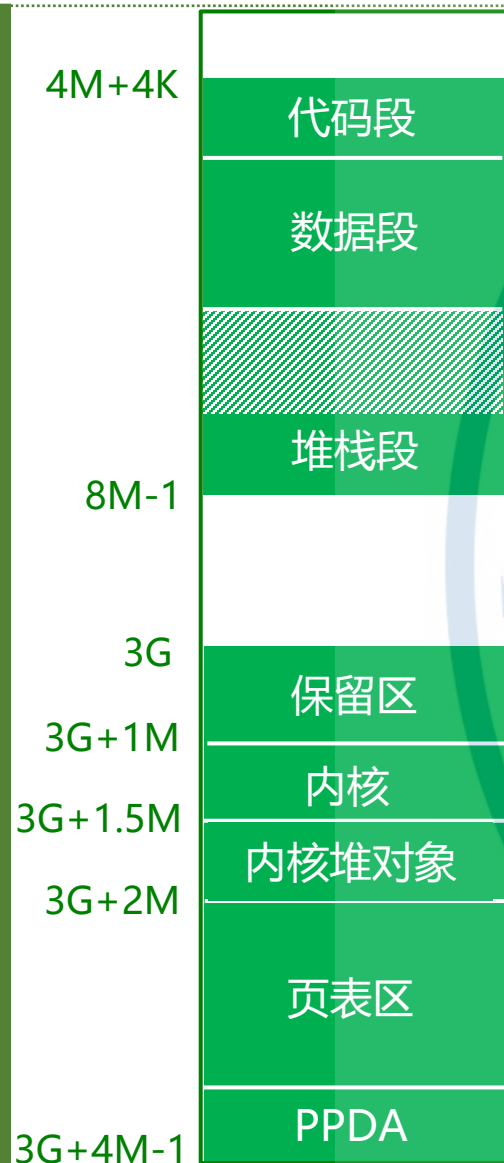
pPageDirectory



UNIX V6++的地址变换



页目录和核心页表的初始化



```
void Machine::InitPageDirectory()
{
    PageDirectory* pPageDirectory = (PageDirectory*)
        (PAGE_DIRECTORY_BASE_ADDRESS + KERNEL_SPACE_START_ADDRESS);
    .....;
    /* 填写200#页表的第768项 */
    unsigned int kPageTableIdx = KERNEL_SPACE_START_ADDRESS /
        PageTable::SIZE_PER_PAGETABLE_MAP;
    pPageDirectory->m_Entrys[kPageTableIdx].m_UserSupervisor = 0;
    pPageDirectory->m_Entrys[kPageTableIdx].m_Present = 1;
    pPageDirectory->m_Entrys[kPageTableIdx].m_ReadWriter = 1;
    pPageDirectory->m_Entrys[kPageTableIdx].m_PageTableBaseAddress =
        KERNEL_PAGE_TABLE_BASE_ADDRESS >> 12;
```

3G/4M

0x201

Page Directory (0x200号页框)

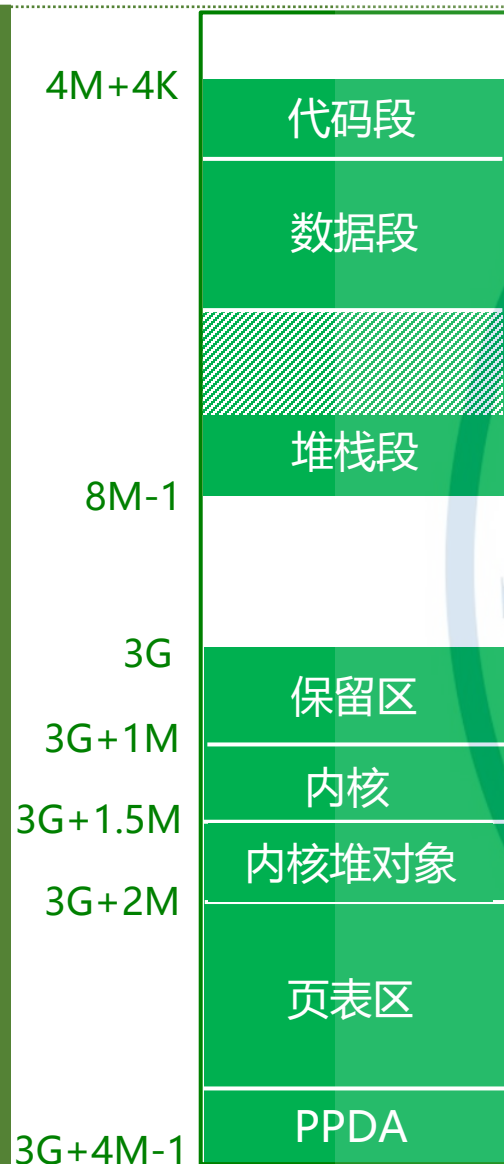
	Page Base Address	s/u	r/w	p
0#				
1#				
.....				
768#	0x201	0	1	1
.....				



UNIX V6++的地址变换



页目录和核心页表的初始化



```
void Machine::InitPageDirectory()
{
    PageDirectory* pPageDirectory = (PageDirectory*)
        (PAGE_DIRECTORY_BASE_ADDRESS + KERNEL_SPACE_START_ADDRESS);
    .....;
    /* 填写200#页表的第768项 */
    unsigned int kPageTableIdx = KERNEL_SPACE_START_ADDRESS /
        PageTable::SIZE_PER_PAGETABLE_MAP;

    pPageDirectory->m_Entrys[kPageTableIdx].m_UserSupervisor = 0;
    pPageDirectory->m_Entrys[kPageTableIdx].m_Present = 1;
    pPageDirectory->m_Entrys[kPageTableIdx].m_ReadWriter = 1;
    pPageDirectory->m_Entrys[kPageTableIdx].m_PageTableBaseAddress =
        KERNEL_PAGE_TABLE_BASE_ADDRESS >> 12;

    PageTable* pPageTable = (PageTable*)
        (KERNEL_PAGE_TABLE_BASE_ADDRESS + KERNEL_SPACE_START_ADDRESS);
```

0x201000

3G

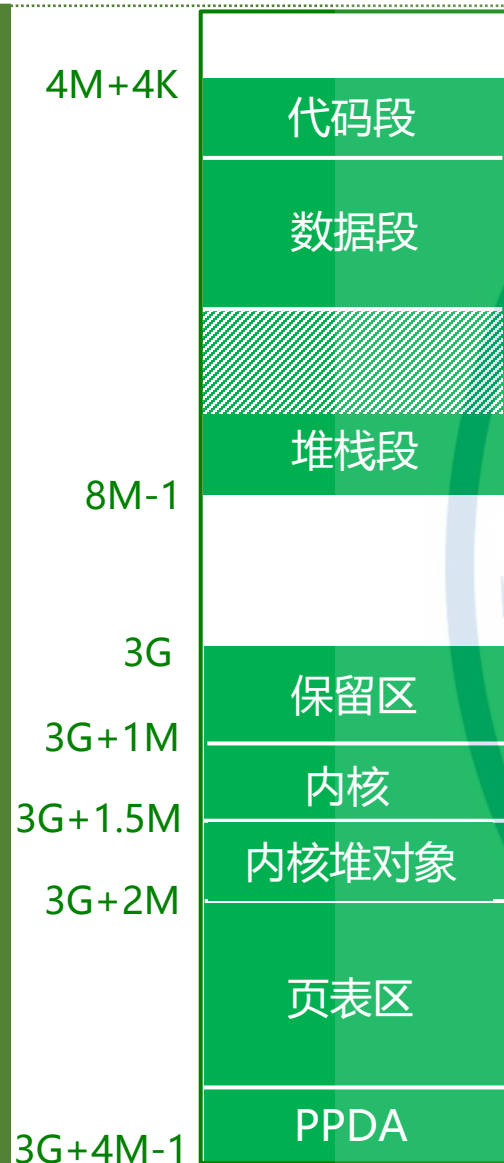
pPageTable



UNIX V6++的地址变换



页目录和核心页表的初始化



```
void Machine::InitPageDirectory()
{
    PageDirectory* pPageDirectory = (PageDirectory*)
        (PAGE_DIRECTORY_BASE_ADDRESS + KERNEL_SPACE_START_ADDRESS);
    .....;
    /* 填写200#页表的第768项 */
    unsigned int kPageTableIdx = KERNEL_SPACE_START_ADDRESS /
        PageTable::SIZE_PER_PAGETABLE_MAP;

    pPageDirectory->m_Entrys[kPageTableIdx].m_UserSupervisor = 0;
    pPageDirectory->m_Entrys[kPageTableIdx].m_Present = 1;
    pPageDirectory->m_Entrys[kPageTableIdx].m_ReadWriter = 1;
    pPageDirectory->m_Entrys[kPageTableIdx].m_PageTableBaseAddress =
        KERNEL_PAGE_TABLE_BASE_ADDRESS >> 12;

    PageTable* pPageTable = (PageTable*)
        (KERNEL_PAGE_TABLE_BASE_ADDRESS + KERNEL_SPACE_START_ADDRESS);
    for ( unsigned int i = 0; i < PageTable::ENTRY_CNT_PER_PAGETABLE; i++ )
    {
        pPageTable->m_Entrys[i].m_UserSupervisor = 0;
        pPageTable->m_Entrys[i].m_Present = 1;
        pPageTable->m_Entrys[i].m_ReadWriter = 1;
        pPageTable->m_Entrys[i].m_PageBaseAddress = i;
    }
    this->m_PageDirectory = pPageDirectory;
    this->m_KernelPageTable = pPageTable;
}
```

Page Table 768# (0x201号页框)

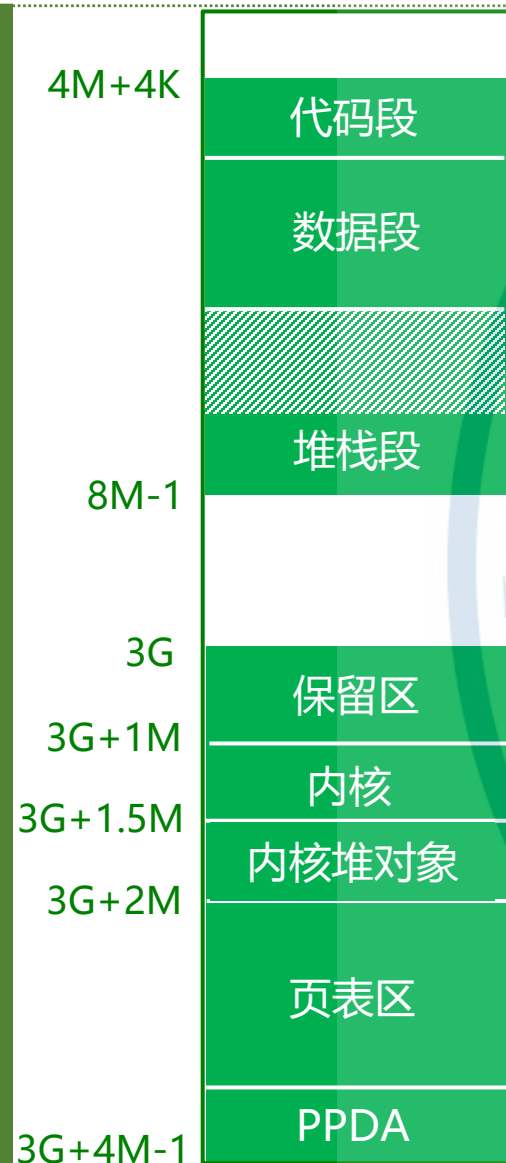
	Page Base Address	s/u	r/w	p
0#	0	0	1	1
1#	1	0	1	1
.....				
1022#	1022	0	1	1
1023#	1023	0	1	1



UNIX V6++的地址变换



用户页表的初始化



```
void Machine::InitUserPageTable()
```

```
{
```

```
PageDirectory* pPageDirectory = this->m_PageDirectory;
```

```
PageTable* pUserPageTable = (PageTable*)
```

```
(USER_PAGE_TABLE_BASE_ADDRESS + KERNEL_SPACE_START_ADDRESS);
```

0x202000

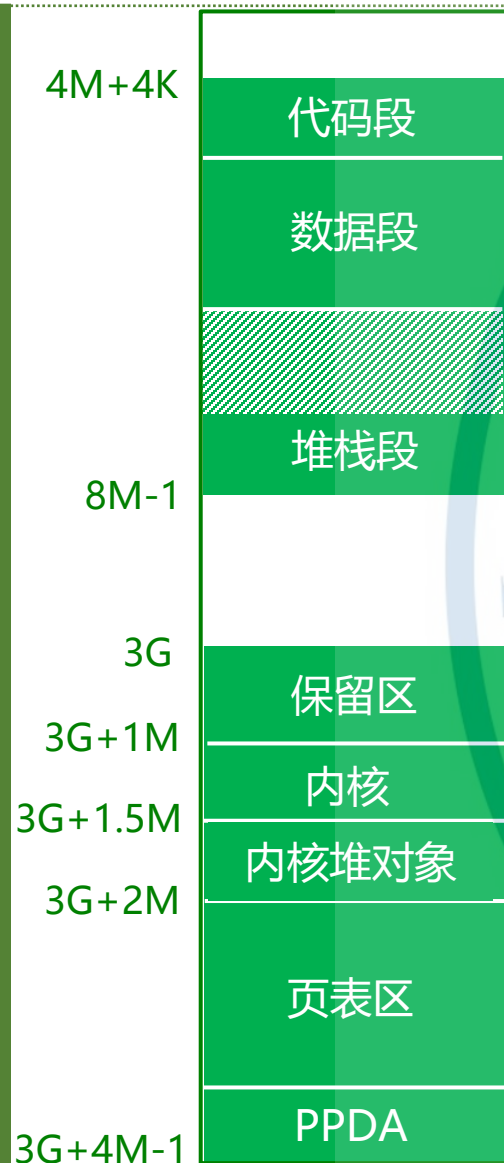
3G



UNIX V6++的地址变换



用户页表的初始化



```
void Machine::InitUserPageTable()
```

```
{
```

```
PageDirectory* pPageDirectory = this->m_PageDirectory;
```

```
PageTable* pUserPageTable = (PageTable*)
```

```
(USER_PAGE_TABLE_BASE_ADDRESS + KERNEL_SPACE_START_ADDRESS);
```

```
unsigned int idx = USER_PAGE_TABLE_BASE_ADDRESS >> 12; 0x202
```

```
for ( unsigned int j = 0; j < USER_PAGE_TABLE_CNT; j++, idx++ )
```

```
{
```

```
pPageDirectory->m_Entrys[j].m_UserSupervisor = 1;
```

```
pPageDirectory->m_Entrys[j].m_Present = 1;
```

```
pPageDirectory->m_Entrys[j].m_ReadWriter = 1;
```

```
pPageDirectory->m_Entrys[j].m_PageTableBaseAddress = idx;
```

```
}
```

```
}
```

Page Directory (0x200号页框)

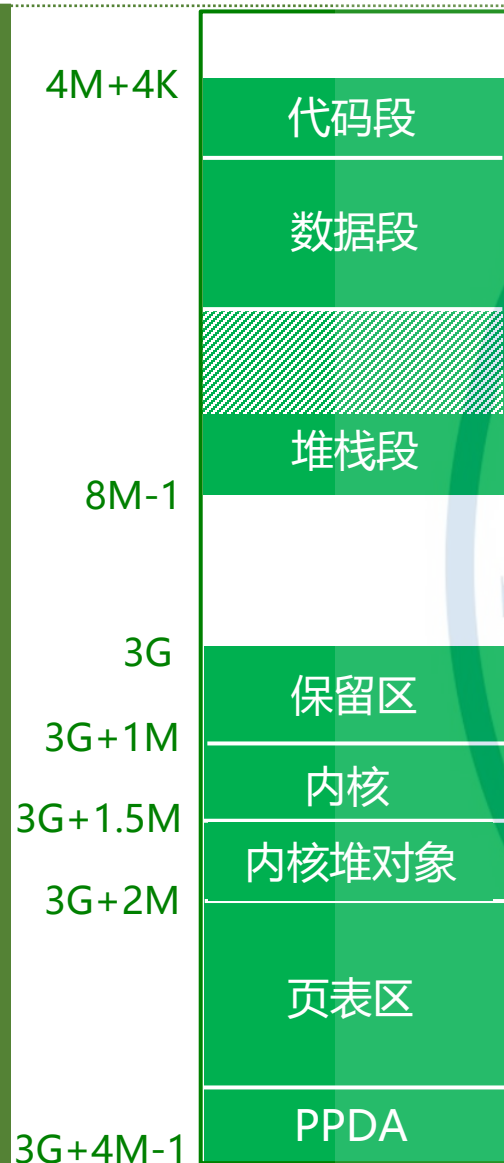
	Page Base Address	s/u	r/w	p
0#	0x202	1	1	1
1#	0x203	1	1	1
.....				
768#	0x201	0	1	1
.....				



UNIX V6++的地址变换



用户页表的初始化



```
void Machine::InitUserPageTable()
{
    PageDirectory* pPageDirectory = this->m_PageDirectory;
    PageTable* pUserPageTable = (PageTable*)
        (USER_PAGE_TABLE_BASE_ADDRESS + KERNEL_SPACE_START_ADDRESS);

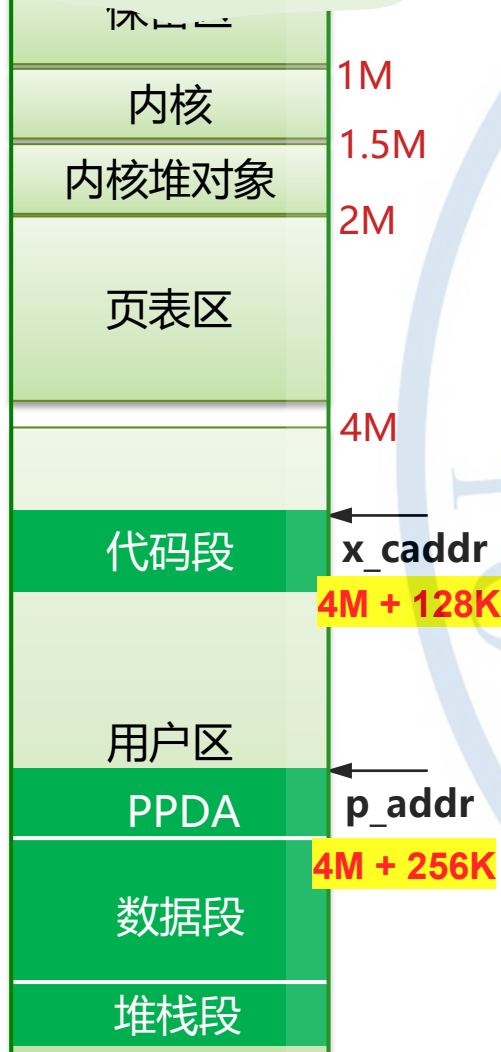
    unsigned int idx = USER_PAGE_TABLE_BASE_ADDRESS >> 12;
    for ( unsigned int j = 0; j < USER_PAGE_TABLE_CNT; j++, idx++ )
    {
        pPageDirectory->m_Entrys[j].m_UserSupervisor = 1;
        pPageDirectory->m_Entrys[j].m_Present = 1;
        pPageDirectory->m_Entrys[j].m_ReadWriter = 1;
        pPageDirectory->m_Entrys[j].m_PageTableBaseAddress = idx;
        // 1024
        for ( unsigned int i = 0; i < PageTable::ENTRY_CNT_PER_PAGETABLE; i++ )
        {
            pUserPageTable[j].m_Entrys[i].m_UserSupervisor = 1;
            pUserPageTable[j].m_Entrys[i].m_Present = 1;
            pUserPageTable[j].m_Entrys[i].m_ReadWriter = 1;
            pUserPageTable[j].m_Entrys[i].m_PageBaseAddress = 0x00000 + i + j * 1024;
        }
    }
    this->m_UserPageTable = pUserPageTable;
}
```



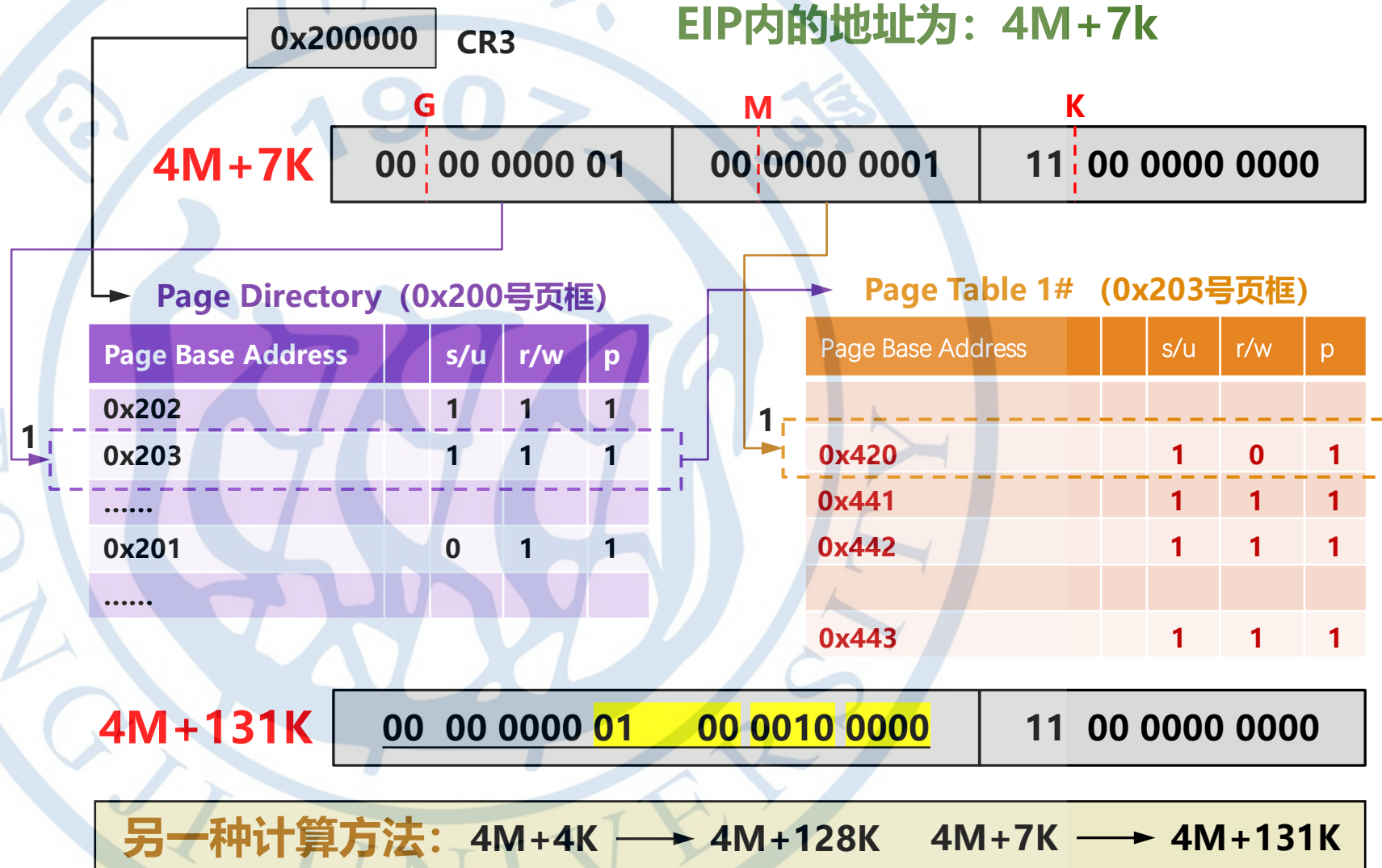
UNIX V6++的地址变换



进程取指时:



EIP内的地址为: $4M + 7K$

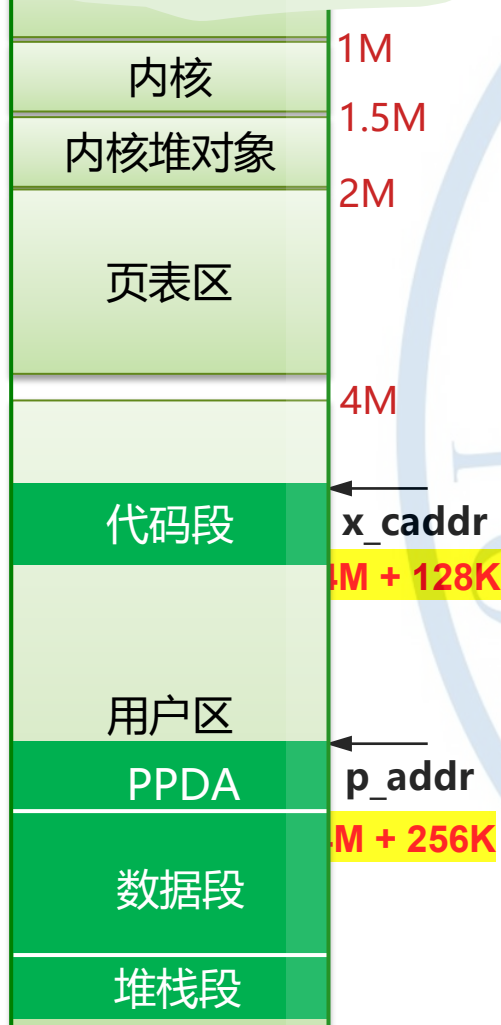




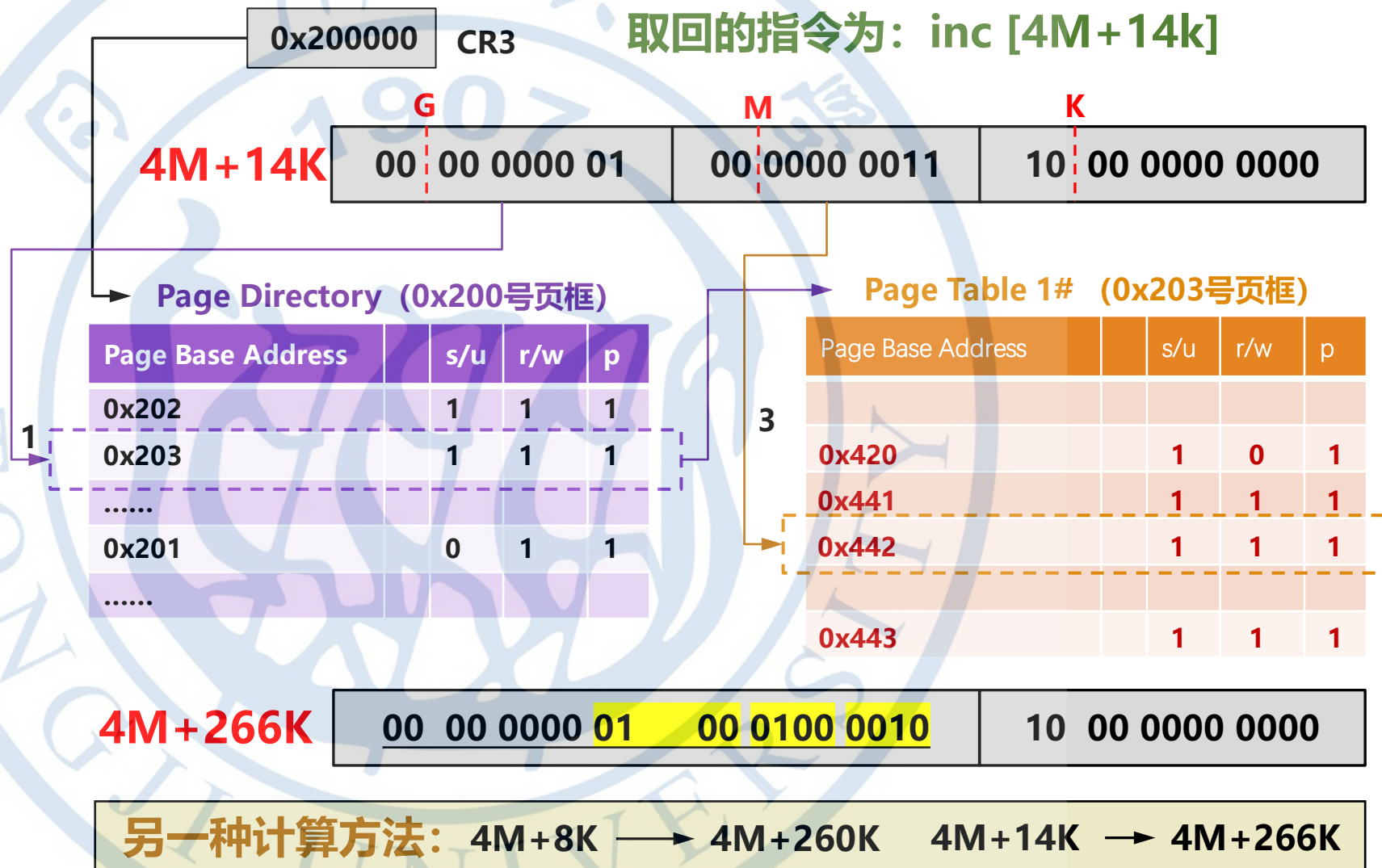
UNIX V6++的地址变换



执行指令时:



取回的指令为: `inc [4M+14k]`





本节小结



- 1 **UNIX V6++中进程核心态与用户态下的逻辑地址空间**
- 2 **UNIX V6++中进程核心态与用户态下的物理地址空间**
- 3 **UNIX V6++中利用两级页表实现的地址变换过程**

阅读教材：154页 ~ 164页